

Wydział Elektroniki i Technik Informacyjnych
Politechnika Warszawska

Projektowanie układów sterowania
(projekt grupowy)

Sprawozdanie z projektu i ćwiczenia laboratoryjnego
nr 2, zadanie nr 11

Piotr Patek, Kamil Cisek, Mateusz Smardzewski, Piotr
Sienkiewicz

Warszawa, 2024

Spis treści

1. Projekt	2
1.1. Identyfikacja procesu	2
1.1.1. Wyjście procesu w punkcie pracy	2
1.1.2. Odpowiedzi skokowe	3
1.1.3. Charakterystyka statyczna $y(u, z)$	4
1.1.4. Odpowiedzi skokowe dla algorytmu DMC	5
1.2. Analityczny algorytm DMC z pomiarem zakłócenia	5
1.2.1. Strojenie regulatora DMC dla skoku jednostkowego i braku zakłócenia	9
1.2.2. Dostrajanie regulatora DMC z pomiarem zakłócenia	14
1.2.3. Układ regulacji z regulatorem DMC przy zakłóceniu sinusoidalnym	17
1.2.4. Testy odporności regulatora DMC z pomiarem zakłócenia na szum pomiarowy	20
2. Laboratorium	28
2.1. Identyfikacja procesu	28
2.1.1. Ustalanie punktu pracy stanowiska	28
2.1.2. Zbieranie odpowiedzi skokowych dla toru zakłócenie $\rightarrow$ wyjście procesu	29
2.1.3. Aproksymacja odpowiedzi skokowych dla toru zakłócenie $\rightarrow$ wyjście procesu za pomocą członu inercyjnego drugiego rzędu z opóźnieniem	32
2.1.4. Otrzymywanie wektora odpowiedzi skokowej dla algorytmu DMC	35
2.1.5. Zbieranie odpowiedzi skokowych dla toru wejście $\rightarrow$ wyjście procesu oraz aproksymacja za pomocą członu inercyjnego drugiego rzędu z opóźnieniem	39
2.2. Analityczny algorytm DMC z pomiarem zakłócenia	40
2.2.1. Implementacja algorytmu DMC z pomiarem zakłócenia działającego na otrzymanym modelu stanowiska grzejąco-chłodzącego	40
2.2.2. Testowanie i strojenie regulatora DMC z pomiarem zakłócenia na otrzymanym modelu stanowiska grzejąco-chłodzącego	44
2.2.3. Implementacja algorytmu DMC z pomiarem zakłócenia działającego na rzeczywistym stanowisku laboratoryjnym	50
2.2.4. Testowanie regulatora DMC z pomiarem zakłócenia na rzeczywistym stanowisku laboratoryjnym	54
2.3. Wnioski	57

1. Projekt

Na potrzeby drugiego bloku laboratoryjnego nasz obiekt regulacji, zarówno w części projektowej jak i laboratoryjnej, został rozszerzony o dodatkowe wejście. Nie jest to jednak kolejny sygnał na który mamy wpływ, a zakłócenie - wejście niesterowane.

1.1. Identyfikacja procesu

Pierwszym krokiem jaki wykonano w ramach projektu było zapoznanie się z dostarczonym symulatorem w postaci zakodowanej funkcji środowiska MATLAB. W tym celu w interpreterze wywołano funkcję przedstawioną na listingu 1.1.

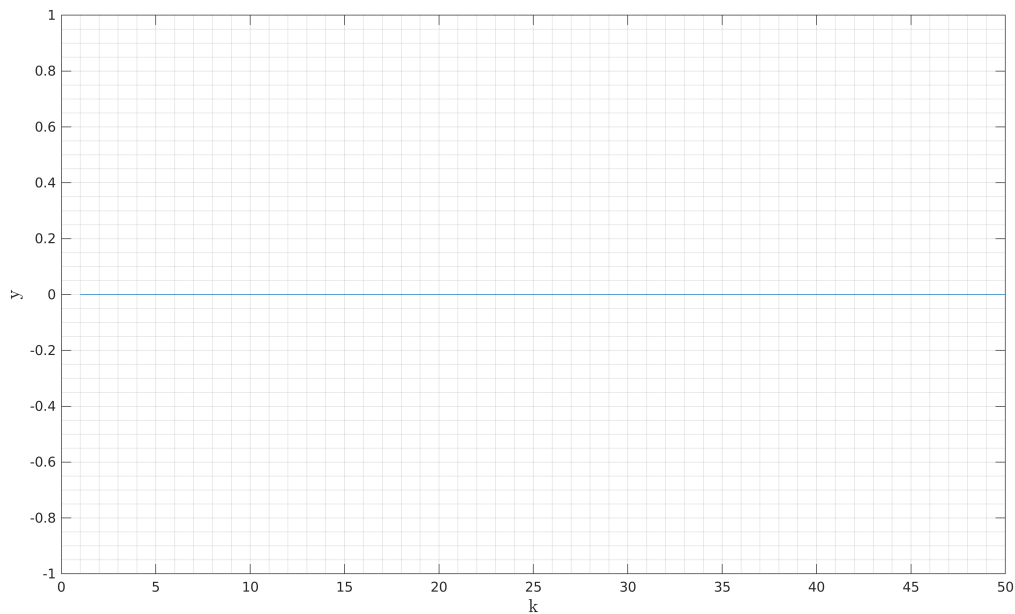
Listing 1.1: Sprawdzenie informacji o obiekcie

```
>> symulacja_obiektu11y_p2
Yk = symulacja_obiektu11y_p2(Ukm7,Ukm8,Zkm3,Zkm4,Ykm1,Ykm2)
Parametry:
  Ukm7 -- u(k-7)
  Ukm8 -- u(k-8)
  Zkm3 -- z(k-3)
  Zkm4 -- z(k-4)
  Ykm1 -- y(k-1)
  Ykm2 -- y(k-2)
Wartość zwracana:
  Yk -- y(k)
```

Jak można zauważyć, obiekt przyjmuje sygnały sterujące u , a także wartości zakłócenia z i wyjścia obiektu y z chwil poprzednich.

1.1.1. Wyjście procesu w punkcie pracy

Zgodnie z poleceniem zweryfikowano wartość punktu pracy dla wyjścia Y_{pp} przy założeniu sygnałów wejściowych równych $U_{pp} = Z_{pp} = 0$.



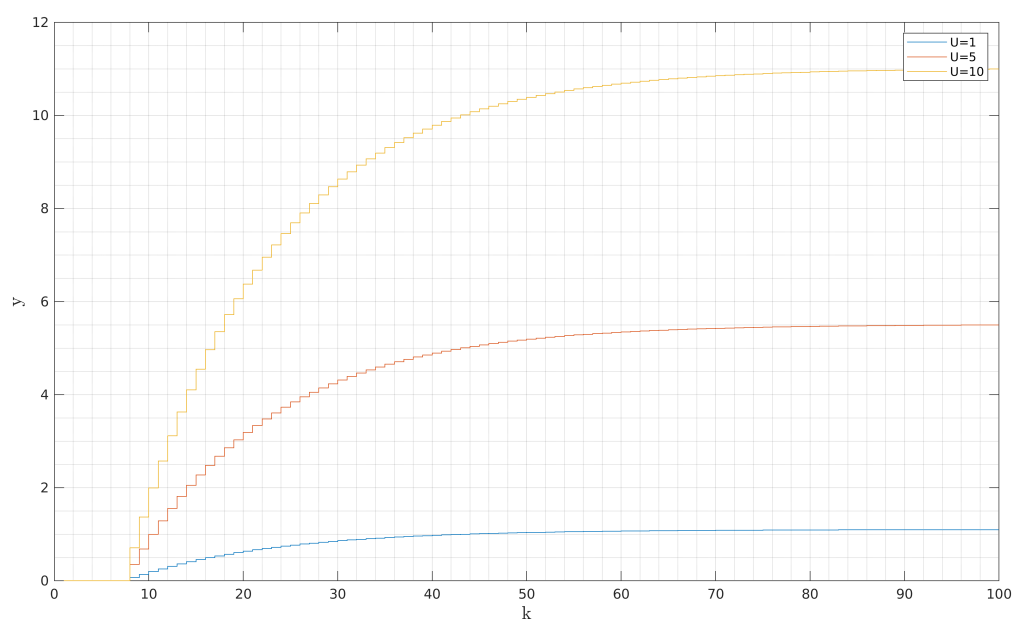
Rys. 1.1: Wyjście procesu y dla sygnału sterującego $U_{pp} = 0$ i zakłócenia $Z_{pp} = 0$

Jak można zauważyć na rys. 1.1 wyjście symulowanego obiektu w punkcie pracy jest zgodne z informacją zawartą w treści zadania.

1.1.2. Odpowiedzi skokowe

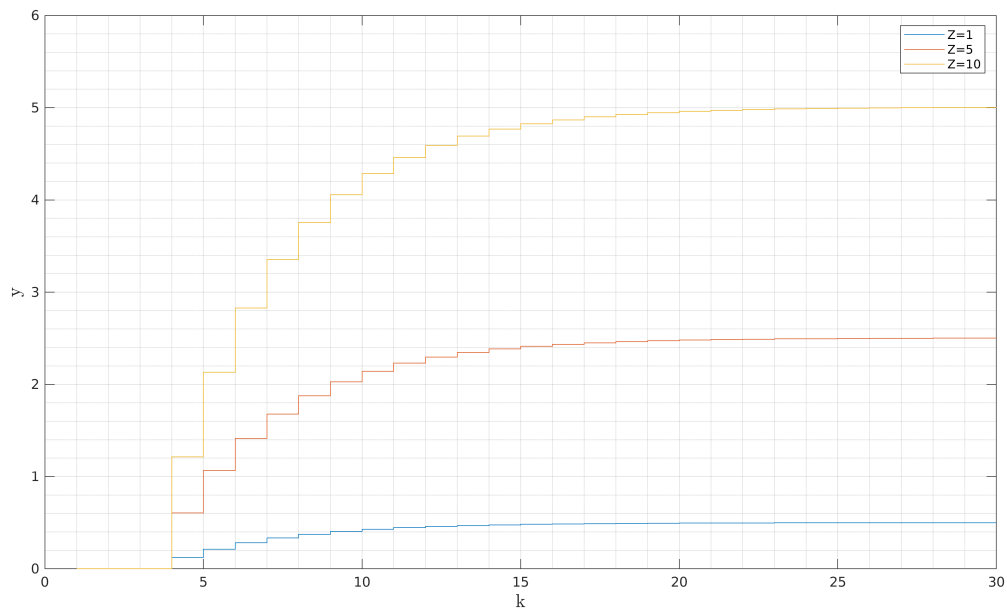
Wykonano trzy skoki sygnału sterowania z wartości $U_{pp} = 0$ do wartości kolejno 1, 5, 10 oraz trzy skoki sygnału zakłócenia do wartości kolejno 1, 5, 10. Odpowiedzi skokowe pokazano na rysunkach 1.2, 1.3

Tor $u \rightarrow y$



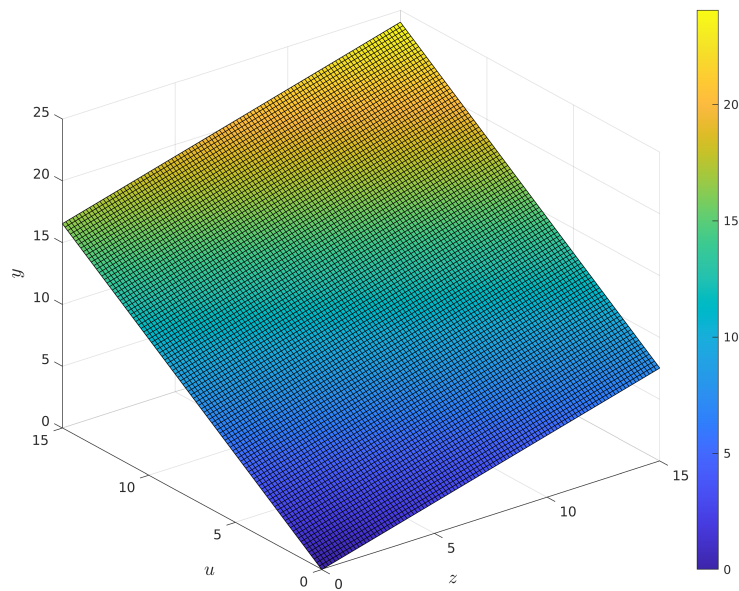
Rys. 1.2: Odpowiedzi skokowe dla toru $u \rightarrow y$

Tor $z \rightarrow y$



Rys. 1.3: Odpowiedzi skokowe dla toru $z \rightarrow y$

1.1.3. Charakterystyka statyczna $y(u, z)$

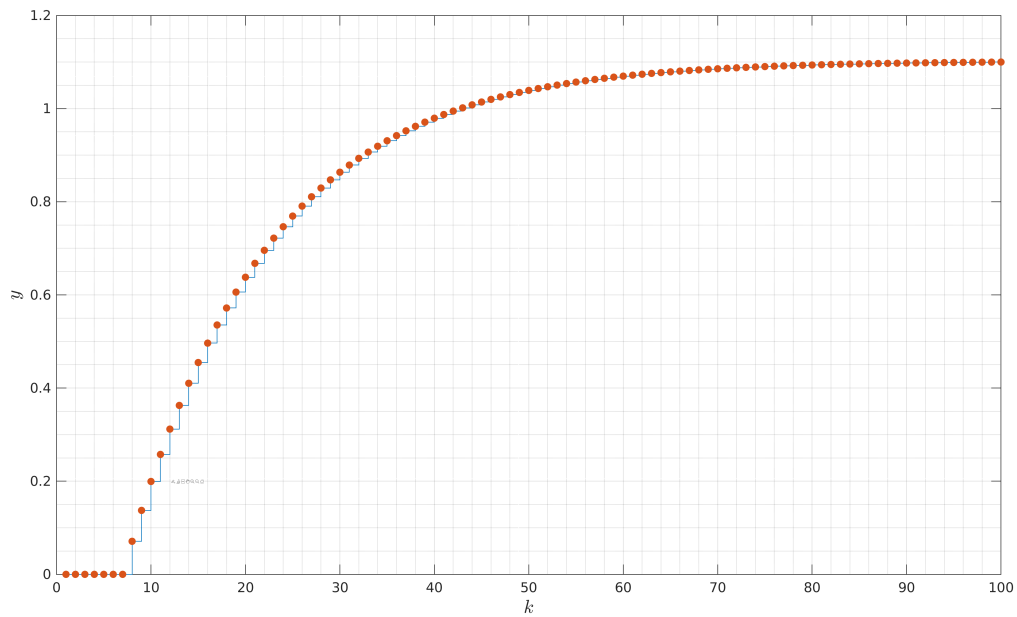


Rys. 1.4: Charakterystyka statyczna $y(u, z)$

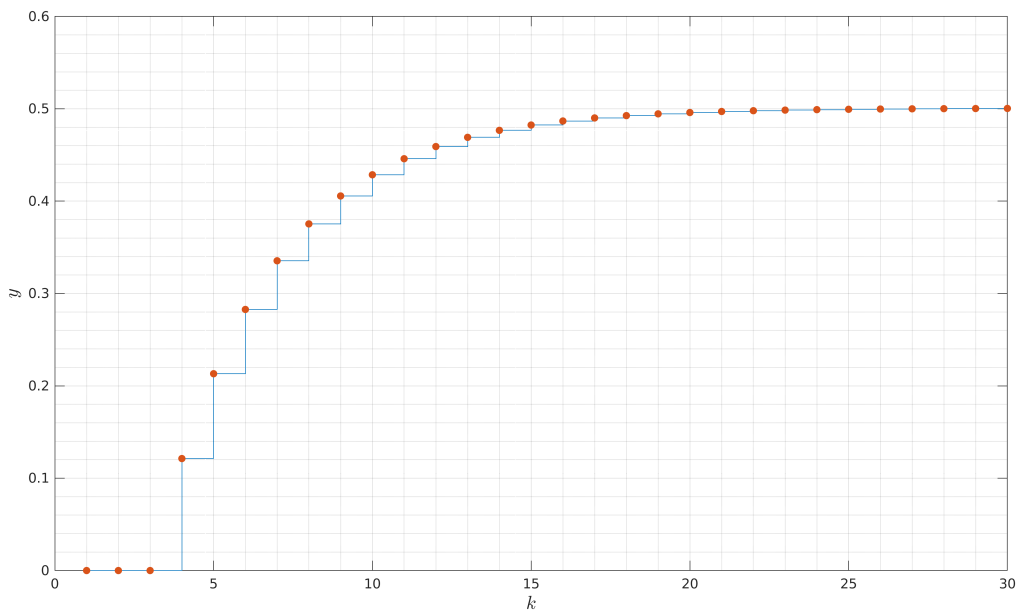
$$K_{\text{stat } u \rightarrow y} = 1.102242$$

$$K_{\text{stat } z \rightarrow y} = 0.500961$$

1.1.4. Odpowiedzi skokowe dla algorytmu DMC



Rys. 1.5: Odpowiedź obiektu na skok jednostkowy w torze $u \rightarrow y$



Rys. 1.6: Odpowiedź obiektu na skok jednostkowy w torze $z \rightarrow y$

1.2. Analityczny algorytm DMC z pomiarem zakłócenia

Do symulacji algorytmu DMC z pomiarem zakłócenia napisano funkcję **dmc_with_disturbance** w języku Matlab (listing 1.2).

Listing 1.2: Regulator DMC z pomiarem zakłócenia

```
function [y, u, z, z_measured] = dmc_with_disturbance(start, kend, Upp, Ypp, N,
Nu, D, lambda, Dz, u_set_time, u_set_value, z, consider_disturbannce, noise)
    %% Adding noise to disturbance measurement
```

```

% Available noise types:
% noise = none - no noise
% noise = gaussian - white Gaussian noise
% noise = uniform - uniform noise
% noise = quantization - quantization noise
% (e.g., introduced by an ADC converter)

z_measured = z;

if noise == "gaussian"
    noise_mean = 0;
    noise_std_dev = 0.2;
    gaussian_noise = noise_mean + noise_std_dev * randn(size(z_measured));
    z_measured = z_measured + gaussian_noise;
elseif noise == "uniform"
    noise_min = -0.1;
    noise_max = 0.1;
    uniform_noise = noise_min + (noise_max - noise_min) * rand(size(z));
    z_measured = z_measured + uniform_noise;
elseif noise == "quantization"
    q = 0.1;
    z_measured = round(z_measured ./ q) .* q;
end

%% Control limitations - in this case, none
du_min = -1e3;
du_max = 1e3;

u_min = -1e3;
u_max = 1e3;

%% Step responses
s = step_response(Upp, Ypp, D+1, 1, 1, "u");
s = s(2:end);

sz = step_response(Upp, Ypp, Dz+1, 1, 1, "z");
sz = sz(2:end);

% Fill M matrix
M = zeros(N, Nu);
for i=1:Nu
    M(i:end,i)=s(1:N-i + 1);
end

% Fill MP matrix
MP = zeros(N, D-1);
for i = 1:N
    for j = 1:D-1
        if i+j <= D
            MP(i, j) = s(i+j) - s(j);
        else
            MP(i, j) = s(D) - s(j);
        end
    end
end

% Fill MZP matrix
MZP = zeros(N, Dz);
for j = 1:Dz
    for i=1:N

```

```

        if j == 1
            if i <= Dz
                MZP(i, j) = sz(i);
            else
                MZP(i, j) = sz(Dz);
            end
        else
            if i + j - 1 <= Dz
                MZP(i, j) = sz(i+j-1) - sz(j - 1);
            else
                MZP(i, j) = sz(Dz) - sz(j - 1);
            end
        end
    end
end

% Regulator parameters
I = eye(Nu);
K = ((M'*M+lambda*I)^(-1))*M';
Ku = K(1,:)*MP;
Ke = sum(K(1, :));

% Variables initialization
y = ones(kend, 1) * Ypp;
u = ones(kend, 1) * Upp;
deltauk_p = zeros(D-1, 1);
deltaz_p = zeros(Dz, 1);

y_zad(1:u_set_time) = Ypp;
y_zad(u_set_time:kend) = u_set_value;

accumulated_error = 0;
% Main loop
for k=start:kend
    % Generate process output
    y(k) = symulacja_obiektu11y_p2(u(k-7), u(k-8), z(k-3), z(k-4), y(k-1), y(k-2));

    % Compute error
    ek = y_zad(k) - y(k);
    accumulated_error = accumulated_error + ek^2;

    % Disturbance effect
    z_part = 0;
    for j=0:(Dz-1)
        z_part = z_part + K(1, :)*MZP(:, j+1)*deltaz_p(1+j);
    end

    % Compute deltau variable for given control horizon
    if consider_disturbannce
        deltauk = Ke*ek-Ku*deltauk_p- z_part;
    else
        deltauk = Ke*ek-Ku*deltauk_p;
    end

    % Back deltau window
    for n=D-1:-1:2
        deltauk_p(n) = deltauk_p(n-1);
    end

    % Calculate deltaz window
    for n=Dz:-1:2

```



```

        deltaz_p(n) = deltaz_p(n-1);
    end

    if(deltauk < du_min)
        deltauk = du_min;
    elseif(deltauk > du_max)
        deltauk = du_max;
    end

    % Manipulate variable for time k
    u(k) = u(k-1) + deltauk;

    if(u(k) < u_min)
        u(k) = u_min;
    elseif(u(k) > u_max)
        u(k) = u_max;
    end

    % Delta u for time k
    deltauk_p(1) = u(k) - u(k-1);
    deltaz_p(1) = z_measured(k) - z_measured(k-1);
end

fprintf("Error: %f\r\n", accumulated_error);
end

```

Funkcja przyjmuje jako argumenty:

- **start, kend** - ilość iteracji,
- **Upp, Ypp** - punkt pracy,
- **N** - horyzont predykcji,
- **Nu** - horyzont sterowania,
- **D** - horyzont dynamiki,
- **lambda** - współczynnik kary,
- **Dz** - horyzont dynamiki zakłócenia,
- **u_set_timeu_set_value** - skok jednostkowy,
- **z** - sygnał wejściowy zakłócenia,
- **consider_disturbance** - pomiar zakłócenia (true/false),
- **noise** - rodzaj szumu.

Na początku funkcji znajdują się zdefiniowane szumy: gaussowski, równomierny i kwantyzacji. Dalej znajdują się współczynniki charakterystyczne dla symulowanego procesu, później ustawiamy zadaną trajektorię sterowania. Następnie, korzystając z funkcji `stepResponse` uzyskujemy współczynniki odpowiedzi skokowej potrzebne do realizacji algorytmu DMC. Po czym uzupełniane są macierze M i M^P zgodnie z zasadami realizacji algorytmu DMC, gdzie

$$M = \begin{bmatrix} s_1 & 0 & \dots & 0 \\ s_2 & s_1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ s_N & s_{N-1} & \dots & s_{N-N_u+1} \end{bmatrix} \quad M^P = \begin{bmatrix} s_2 - s_1 & s_3 - s_2 & \dots & s_D - s_{D-1} \\ s_3 - s_1 & s_4 - s_2 & \dots & s_{D+1} - s_{D-1} \\ \vdots & \vdots & \ddots & \vdots \\ s_{N+1} - s_1 & s_{N+2} - s_2 & \dots & s_{N+D-1} - s_{D-1} \end{bmatrix}$$

Po tym kroku wyznaczamy macierz K , gdzie

$$K = (M^T \Psi M + \Lambda)^{-1} M^T \Psi$$

i wyznaczamy k^u i k^e . Ostatnim krokiem przed pętlą główną programu jest inicjalizacja wektora wyjść, wejść, wektora ΔU^P i aktualizacja współczynnika jakości - sumy kwadratów błędów.

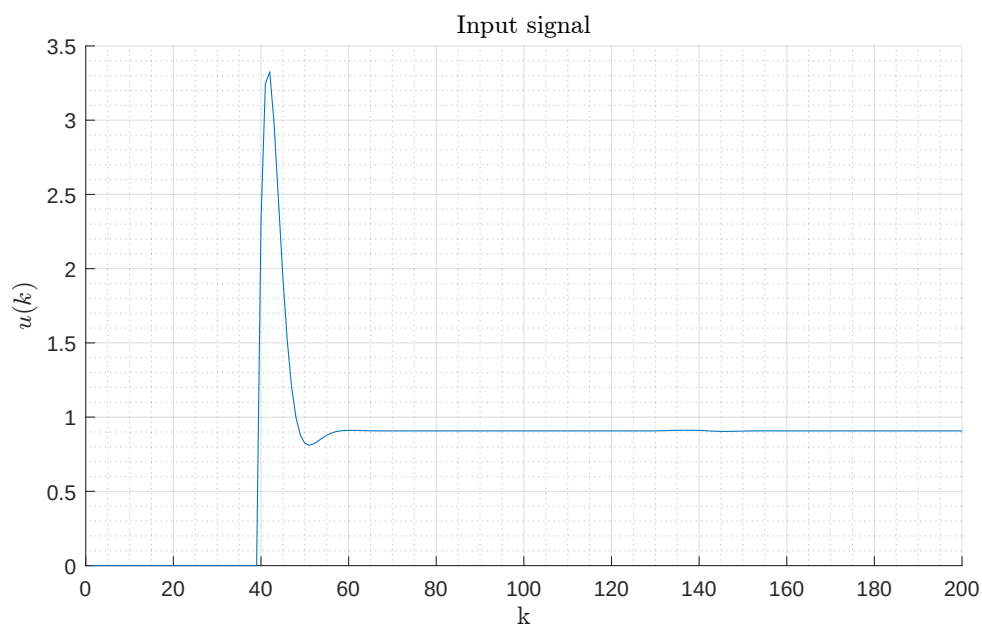
W pętli głównej programu najpierw dokonujemy symulacji obiektu, którego wyjście służy nam do obliczenia aktualnego błędu. Następnie korzystając z analitycznego prawa regulacji algorytmu DMC z pomiarem zakłóceń:

$$\Delta u(k) = \Delta u(k | k) = \sum_{p=1}^N k_{1,p} (y^{\text{zad}}(k) - y(k)) - \sum_{j=1}^{D-1} k_j^u \Delta u(k-j) - \sum_{j=0}^{D-1} k_{j+1}^z \Delta z(k-j)$$

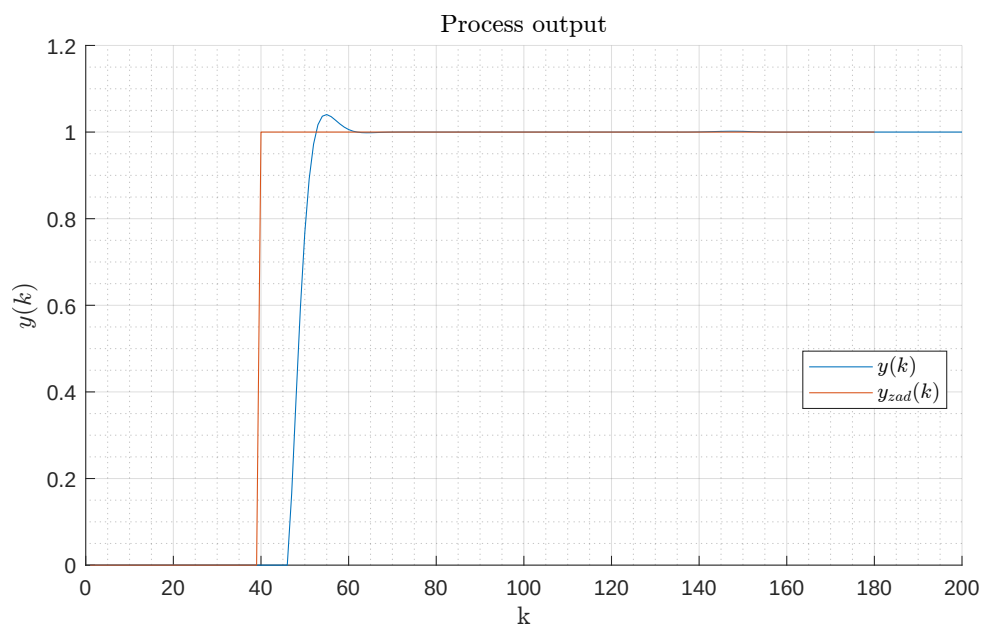
obliczamy aktualny przyrost sterowania. Pozostaje nam jeszcze sprawdzić czy obliczony przyrost spełnia ograniczenia maksymalnego przyrostu sygnału sterującego i czy obliczona wartość sterowania nie jest poza ograniczeniami i możemy dodać nowy element do wektora przeszłych przyrostów sterowania.

1.2.1. Strojenie regulatora DMC dla skoku jednostkowego i braku zakłócenia

Jako że z rysunku 1.5 można się dowiedzieć, że obiekt osiąga stan ustalony w czasie około 100 iteracji (lub trochę mniejszym) po wykonaniu skoku sygnału sterującego $u(k)$, zgodnie ze sztuką strojenia regulatorów DMC, rozpoczęto testowanie algorytmu od parametrów $D = N = N_u = 100$ oraz przyjęto $\lambda = 0, 1$. Na wykresach 1.7 i 1.8 przedstawione zostało zachowanie algorytmu dla tych parametrów. Regulator zapewnił w tym przypadku dobrą jakość regulacji, jednakże jest ona okupiona dużą ilością niepotrzebnych obliczeń, dlatego kolejnymi krokami było zmniejszanie kolejnych horyzontów. W przypadku $D = N = N_u = 80$ zauważono nieznaczne pogorszenie się jakości regulacji, dlatego też zdecydowano się na pozostawienie horyzontu dynamiki $D = 100$.

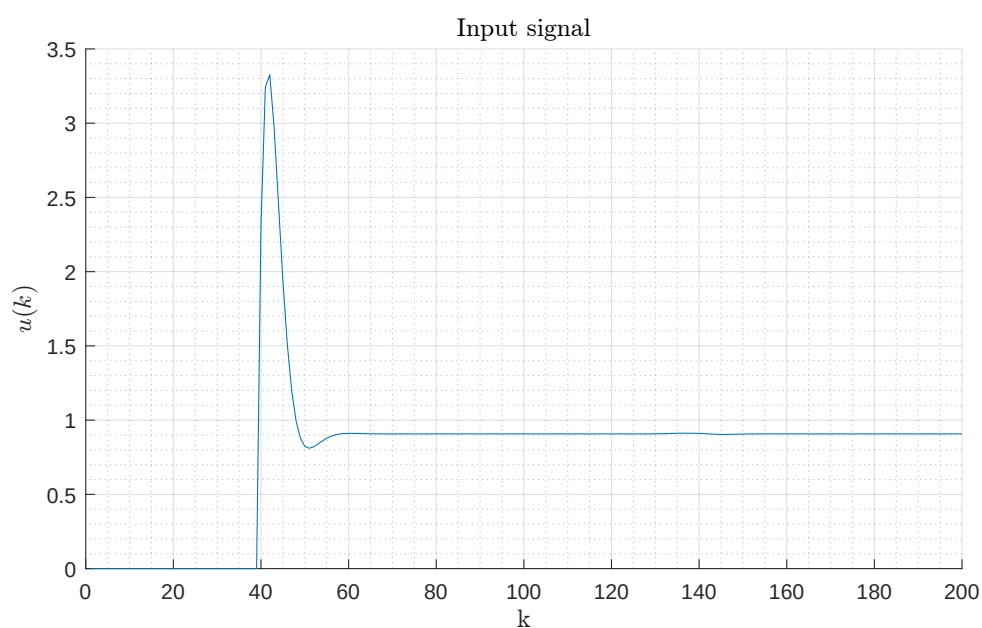


Rys. 1.7: Wykres sygnału wejściowego dla $D = 100$, $N = 100$, $N_u = 100$, $\lambda = 0.1$

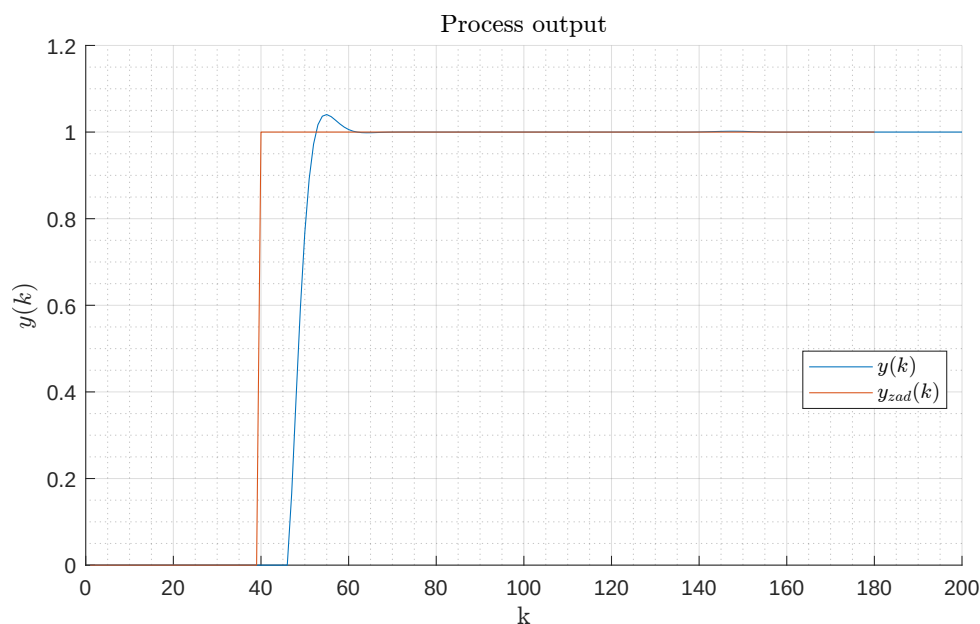


Rys. 1.8: Wykres wyjścia dla $D = 100$, $N = 100$, $N_u = 100$, $\lambda = 0.1$

W następnym kroku zmniejszono razem wartość horyzontów predykcji i sterowania do momentu, aż znowu dało się zauważyć delikatne pogorszenie działania regulatora. Przyjęto więc wartości $N = N_u = 20$, które były tymi wartościami granicznymi. Wykresy $y(k)$ oraz $u(k)$ sporządzone dla tych parametrów zostały przedstawione na rysunkach 1.9 oraz 1.10.

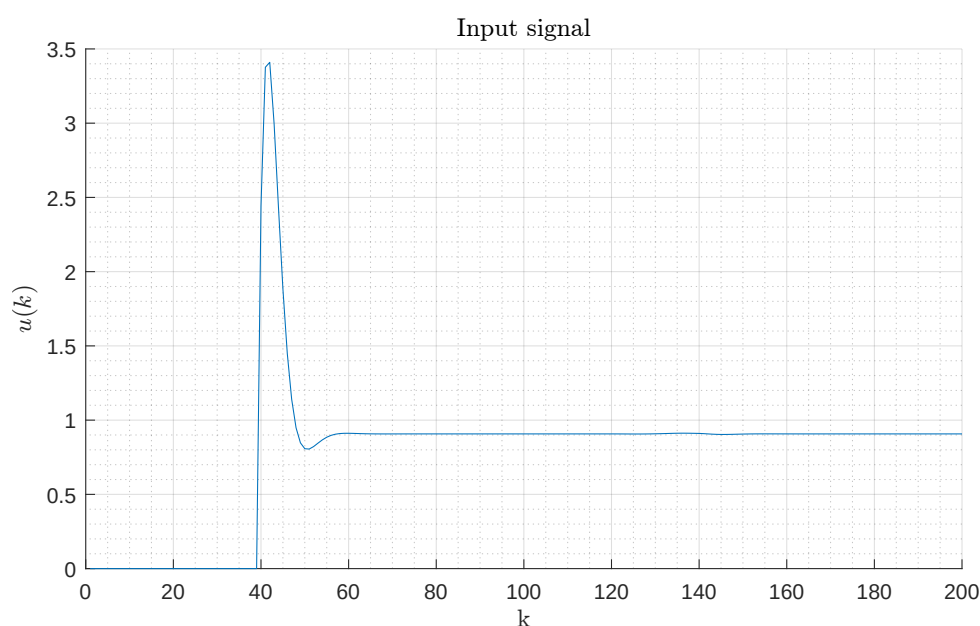


Rys. 1.9: Wykres sygnału wejściowego dla $D = 100$, $N = 20$, $N_u = 20$, $\lambda = 0.1$

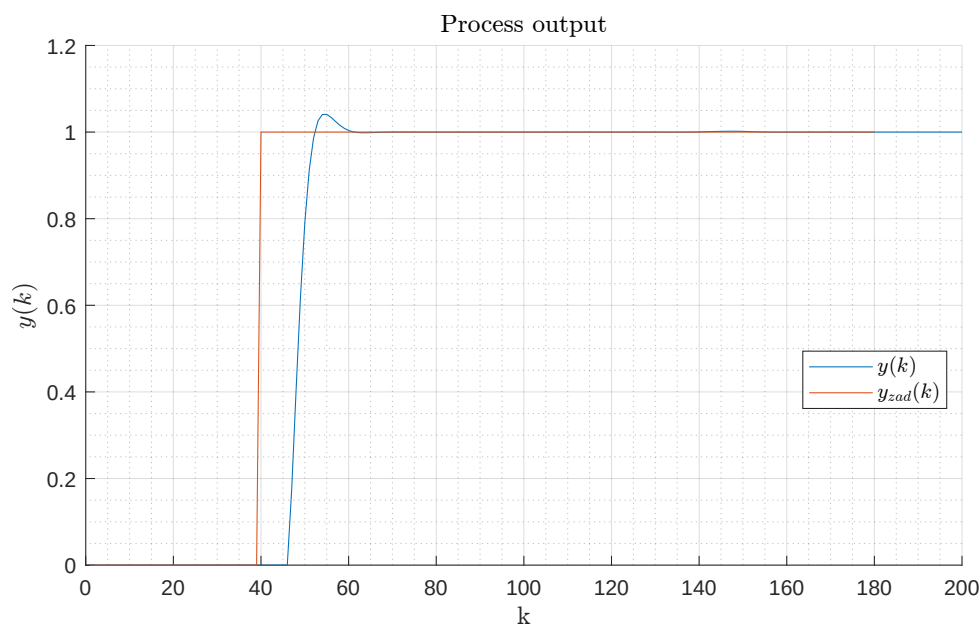


Rys. 1.10: Wykres wyjścia dla $D = 100$, $N = 20$, $N_u = 20$, $\lambda = 0.1$

Kolejnym krokiem było zmniejszanie samego horyzontu sterowania. Najniższą wartość tego parametru, przy której nie zauważono jeszcze pogorszenia jakości regulacji było $N_u = 5$. Tak więc przyjęto w ten sposób ostateczny zestaw parametrów horyzontów $D = 100$, $N = 20$, $N_u = 5$, co pozwoliło uzyskać taką samą jakość regulacji co w przypadku wielkich horyzontów, jednak bez konieczności wykonywania aż tylu zbędnych obliczeń. Działanie regulatora o tej konfiguracji zostało pokazane na wykresach 1.11 oraz 1.12.

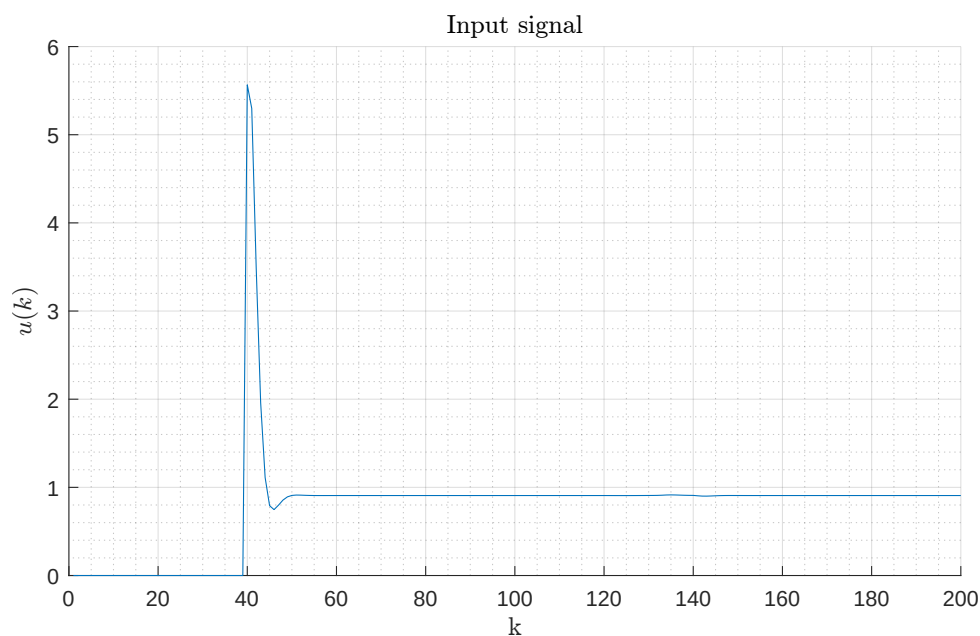


Rys. 1.11: Wykres sygnału wejściowego dla $D = 100$, $N = 20$, $N_u = 5$, $\lambda = 0.1$

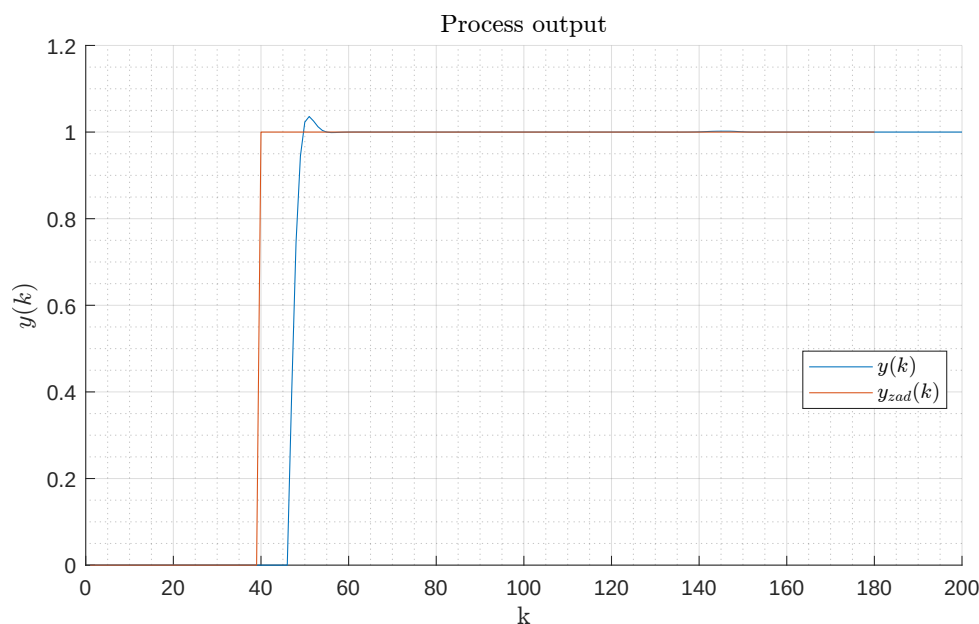


Rys. 1.12: Wykres wyjścia dla $D = 100$, $N = 20$, $N_u = 5$, $\lambda = 0.1$

Aby spróbować jeszcze polepszyć działanie regulatora, ustawiono parametr kary za zbyt szybkie zmiany sygnału sterującego λ na wartość 0,01. Jak to wpłynęło na szybkość i ogólną jakość regulacji, widać na wykresach 1.13 oraz 1.14. W tym przypadku czas ustalania się odpowiedzi obiektu zmalał o kilka iteracji, natomiast przeregulowanie wartości $y(k)$, które i tak było małe, spadło nieznacznie. Jednak nastąpiło to kosztem znacznego wzrostu amplitudy sygnału sterującego, który wcześniej nie przekraczał poziomu 3,4, a teraz dochodzi do wartości 5,5. Tak więc lepszy okazuje się poprzedni zestaw parametrów.



Rys. 1.13: Wykres sygnału wejściowego dla $D = 100$, $N = 20$, $N_u = 5$, $\lambda = 0.01$



Rys. 1.14: Wykres wyjścia dla $D = 100$, $N = 20$, $N_u = 5$, $\lambda = 0.01$

Porównanie wartości błędów regulacji

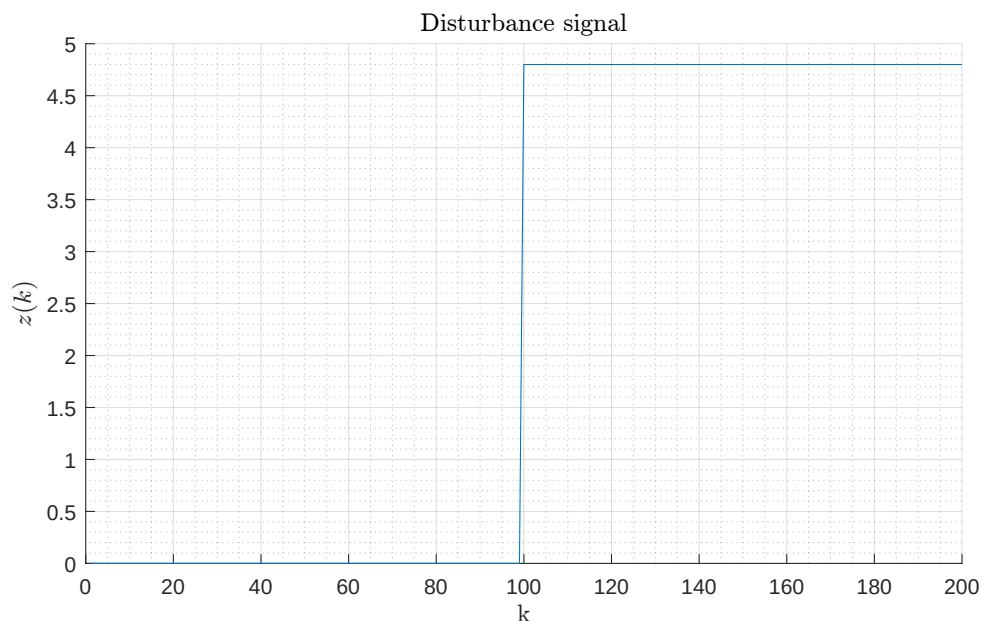
Poniższa tabela przedstawia zestawienie parametrów regulatora, dla których zostały sporządzone powyższe wykresy, a także pokazuje wartości błędu regulacji, który definiowany jest jako suma kwadratów błędów wyliczanych podczas każdej iteracji algorytmu.

Parametry algorytmu DMC	Wartości błędu
$D = 100, N = 100, N_u = 100, \lambda = 0.1$	8.314729
$D = 100, N = 20, N_u = 20, \lambda = 0.1$	8.314670
$D = 100, N = 20, N_u = 2, \lambda = 0.1$	8.246289
$D = 100, N = 20, N_u = 5, \lambda = 0.01$	7.436807

Wybrano wartości $D = 100$, $N = 20$, $N_u = 5$, $\lambda = 0.1$. Zapewniają one optycznie dobrą jakość regulacji, szybkość, bez oscylacji, znaczących przeregulowań. Co prawda dla $\lambda = 0.01$ wartość wyliczonego błędu jest mniejsza, lecz jednak występujące przy tym parametrze zbyt duże zmiany sterowania $u(k)$ dyskredytują tę wartość parametru. Dodatkowo zbyt mała λ przy wystąpieniu zakłóceń pogarsza działanie algorytmu DMC.

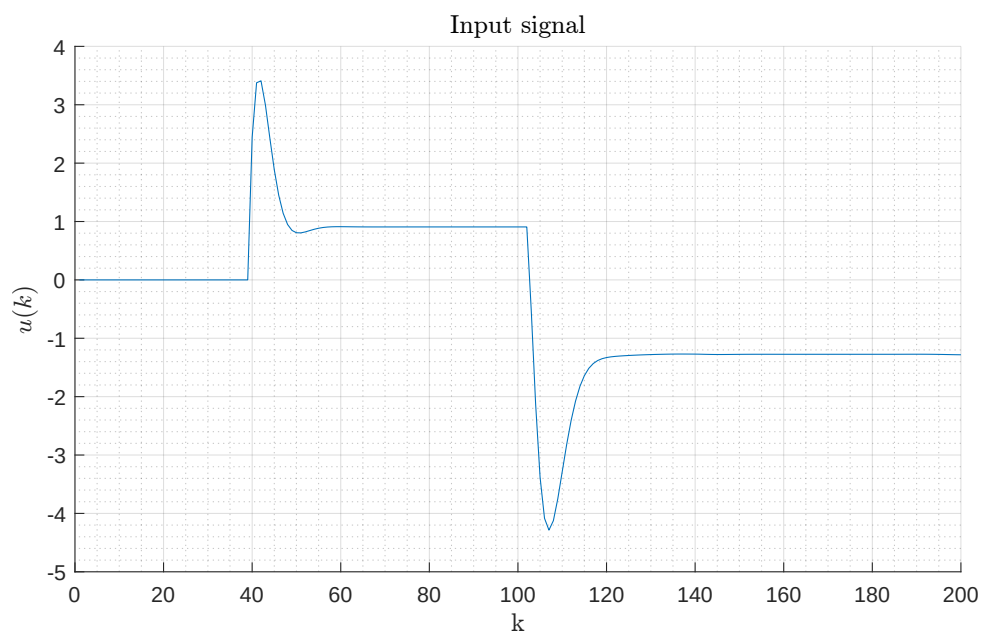
1.2.2. Dostrajanie regulatora DMC z pomiarem zakłócenia

Wykres 1.15 przedstawia przebieg wartości zakłócenia mierzalnego, które było podawane na obiekt w następujących kilku eksperymentach.

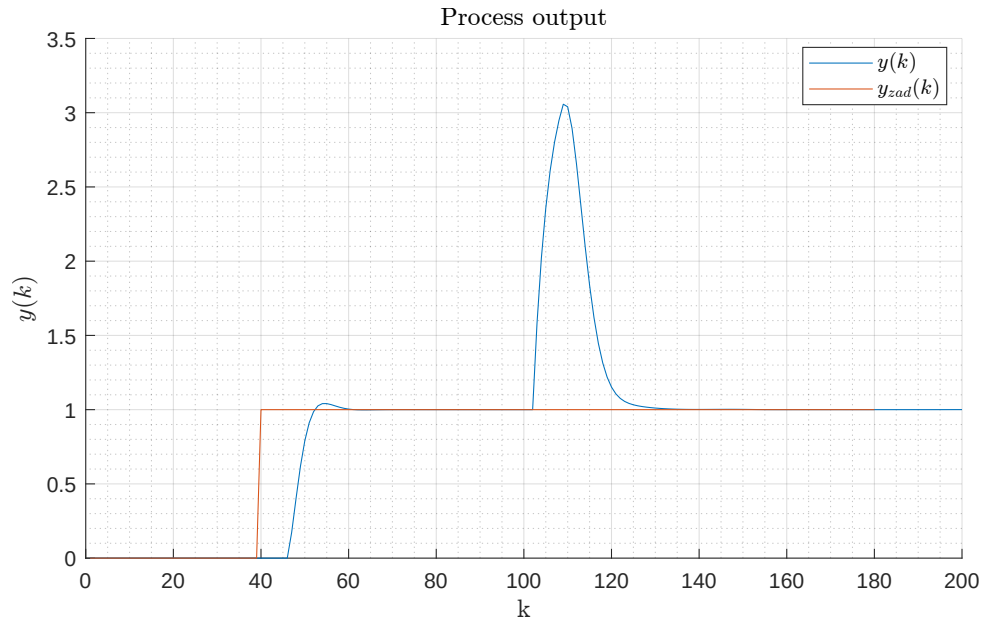


Rys. 1.15: Wykres skoku zakłócenia

Natomiast rysunki 1.16 oraz 1.17 przedstawiają zachowanie algorytmu DMC bez kompensacji zakłócenia, po dodaniu do sygnału wyjściowego $y(k)$ przyczynku wynikającego ze skokowych zmian zakłócenia. Od razu w oczy się rzuca słaba zdolność regulatora do radzenia sobie z zakłóceniami niemierzalnymi. Na wykresie 1.17 widać skok $y(k)$ do wartości przekraczającej 3 razy wartość zadaną. W rzeczywistych obiektach zazwyczaj takie zachowanie regulatora nie byłoby tolerowalne.

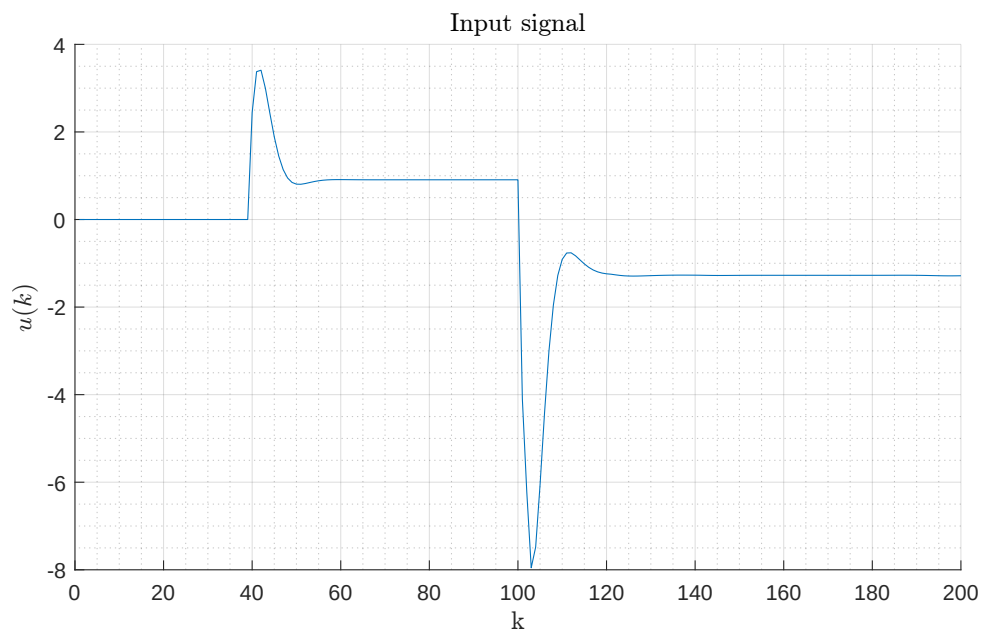


Rys. 1.16: Wykres sygnału wejściowego bez pomiaru zakłócenia dla $D = 100$, $N = 20$, $N_u = 5$, $\lambda = 0.1$

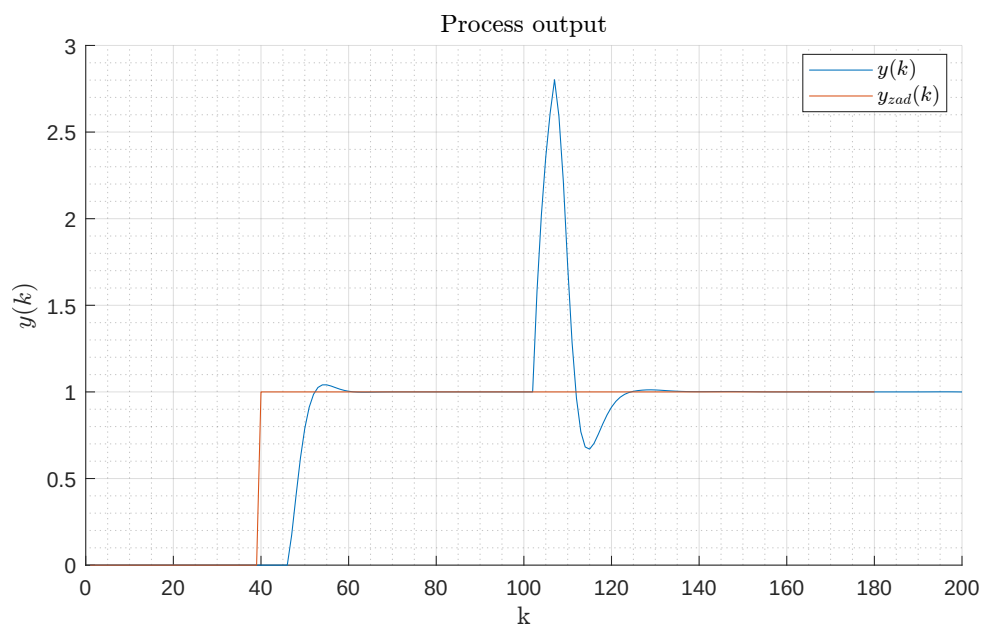


Rys. 1.17: Wykres wyjścia bez pomiaru zakłócenia dla $D = 100$, $N = 20$, $N_u = 5$, $\lambda = 0.1$

Na wykresach 1.18 oraz 1.19 przedstawiono zachowanie regulatora pracującego w tych samych warunkach, jednak z włączoną kompensacją zakłóceń oraz ustawionym horyzontem dynamiki zakłóceń równym $D_z = 20$. Odpowiedź skokowa przedstawiona na wykresie 1.6 jest już względnie w stanie ustalonym dla $k = 20$, a więc taki horyzont dynamiki zdaje się być dobrym punktem wyjścia. Widać, że kompensacja zakłóceń działa, jednak nie przynosi najlepszych rezultatów. Skok $y(k)$ na wyjściu obiektu zmniejszył swoją amplitudę i czas trwania, jednak nadal jest spory. Wzrosły też piki sterowania $u(k)$.

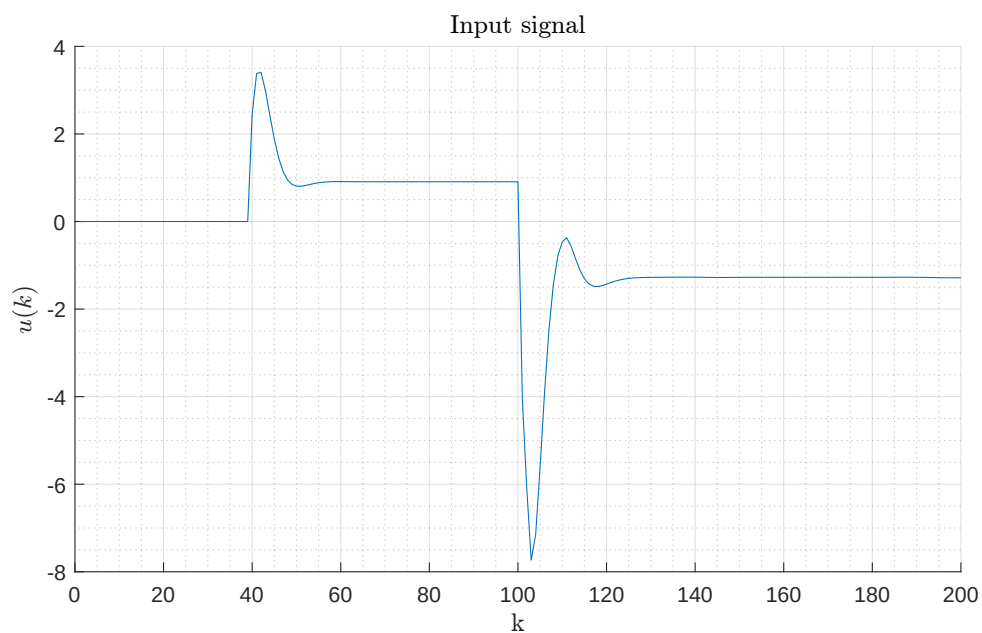


Rys. 1.18: Wykres sygnału wejściowego z pomiarem zakłócenia dla $D = 100$, $N = 20$, $N_u = 5$, $\lambda = 0.1$, $D_z = 20$

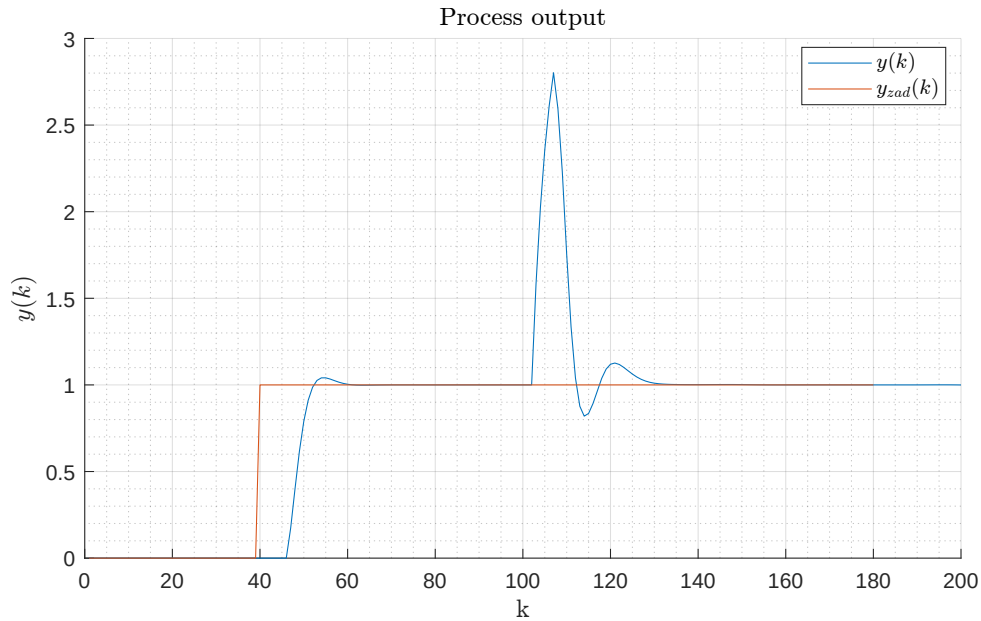


Rys. 1.19: Wykres wyjścia z pomiarem zakłócenia dla $D = 100$, $N = 20$, $N_u = 5$, $\lambda = 0.1$, $D_z = 20$

W następnym kroku zmniejszono horyzont dynamiki zakłócenia do $D_z = 10$, co zmniejszyło ilość wykonywanych przez algorytm obliczeń, nie pogarszając działania algorytmu, a wręcz przeciwnie, nawet delikatnie je poprawiając.



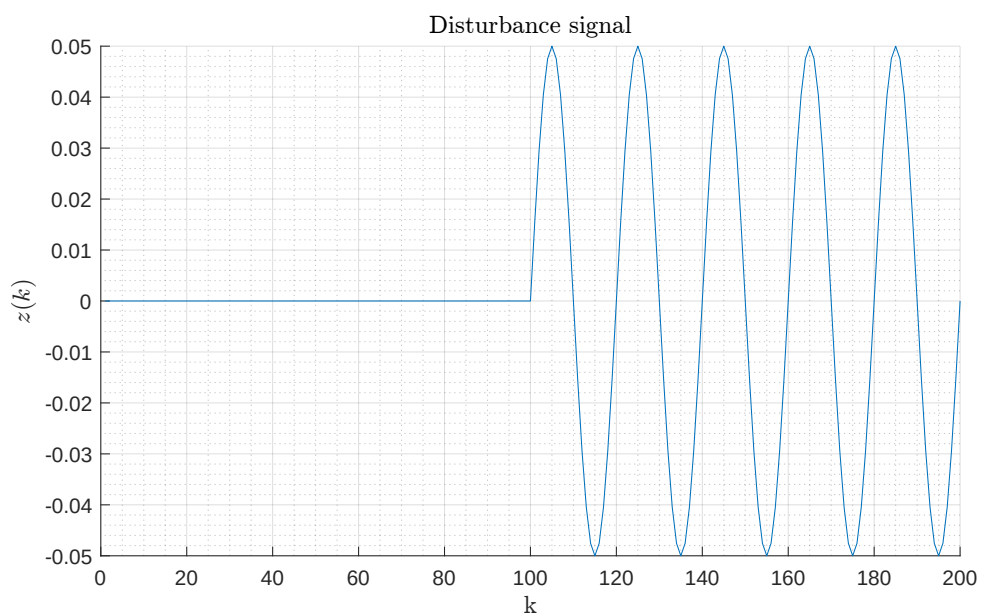
Rys. 1.20: Wykres sygnału wejściowego z pomiarem zakłócenia dla $D = 100$, $N = 20$, $N_u = 5$, $\lambda = 0.1$, $D_z = 10$



Rys. 1.21: Wykres wyjścia z pomiarem zakłócenia dla $D = 100$, $N = 20$, $N_u = 5$, $\lambda = 0.1$, $D_z = 10$

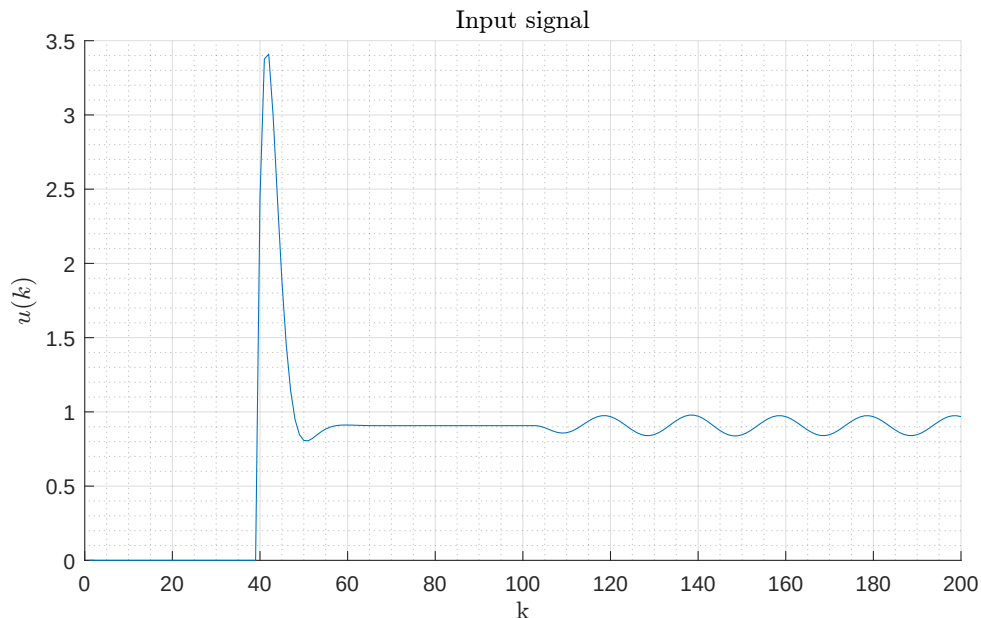
Gdy parametr D_z miał zbyt niską wartość dochodziło do znacznego przeregulowania wyjścia procesu, natomiast wraz z dobraniem odpowiedniej wartości, zwiększanie go nie zmieniało zachowania algorytmu DMC. Na wykresach 1.15 - 1.21 zaobserwowano zachowanie algorytmu DMC z brakiem pomiaru zakłócenia jak i z pomiarem zakłócenia. Dla pierwszego przypadku wskaźnik jakości wyniósł: 40.409902 Dla drugiego przypadku wskaźnik jakości wyniósł: 22.418744 oraz 22.217582. Ostatecznie wskaźnik liczbowy pokazał, że pomiar zakłócenia znacząco wpłynął na jakość regulacji, dodatkowo wyjście procesu było mniej przeregulowane i szybciej osiągało wartość zadaną po wykonaniu skoku zakłócenia.

1.2.3. Układ regulacji z regulatorem DMC przy zakłóceniu sinusoidalnym

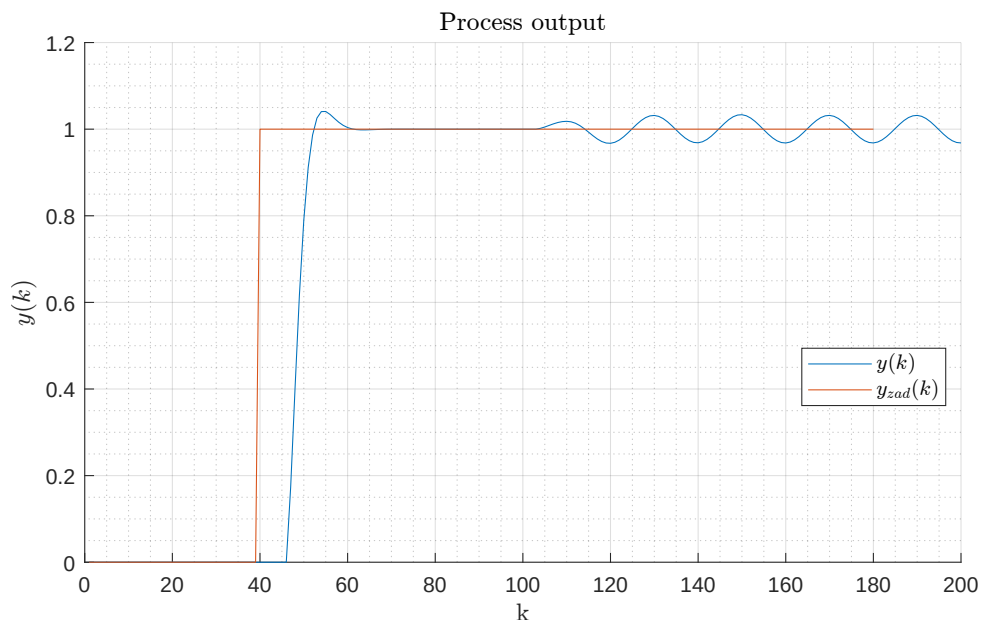


Rys. 1.22: Wykres skoku zakłócenia

W rzeczywistych obiektach jest to bardzo często występujące zakłócenie wynikające z obecności zakłóceń elektrycznych i magnetycznych z sieci energetycznej 50 Hz, a więc jest to symulacja przypominająca realne warunki pracy obiektu. Wykres 1.22 przedstawia sygnał zakłócający sinusoidalny o amplitudzie równej 0,05, natomiast 1.23 oraz 1.24 pracę regulatora DMC w wersji bez kompensacji zakłóceń.

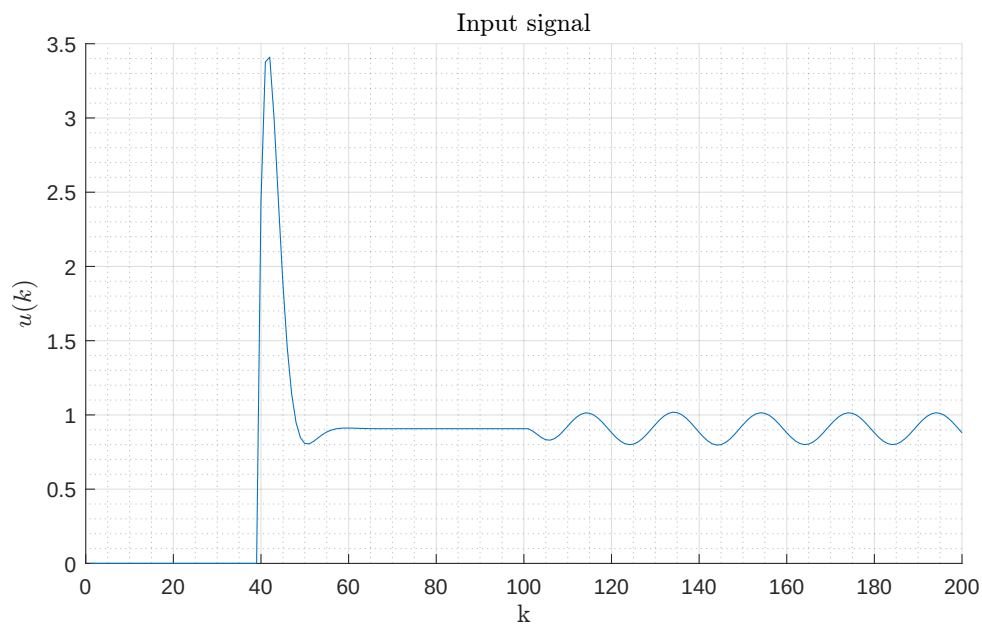


Rys. 1.23: Wykres sygnału wejściowego bez pomiaru zakłócenia dla $D = 100$, $N = 20$, $N_u = 5$, $\lambda = 0.1$

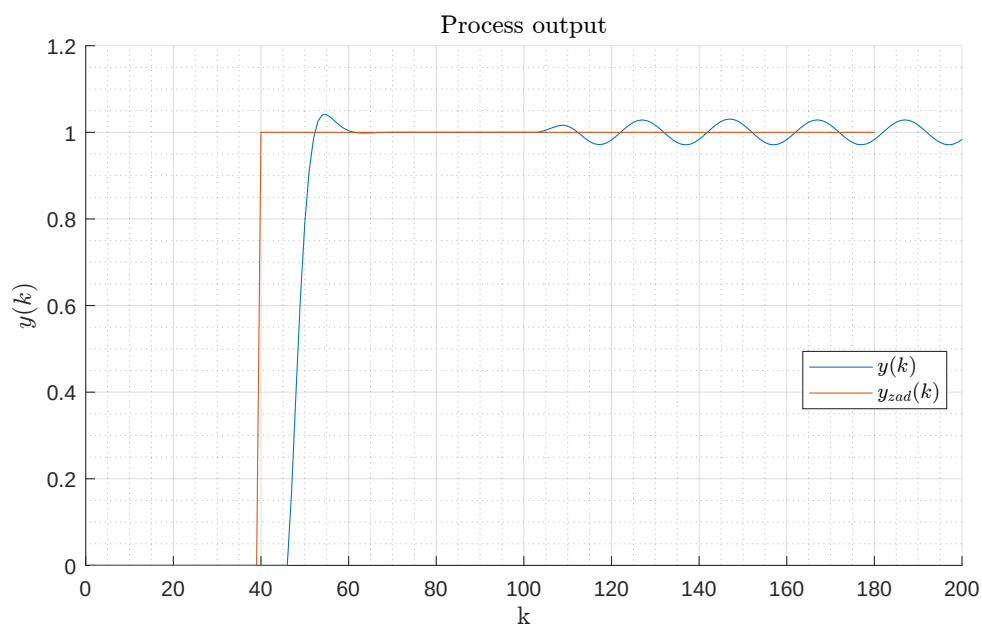


Rys. 1.24: Wykres wyjścia bez pomiaru zakłócenia dla $D = 100$, $N = 20$, $N_u = 5$, $\lambda = 0.1$

Wykresy 1.25 oraz 1.26 pokazują działanie regulatora pracującego przy obecności tych samych zakłóceń, lecz tym razem wykorzystuje ich pomiary.



Rys. 1.25: Wykres sygnału wejściowego z pomiarem zakłócenia dla $D = 100$, $N = 20$, $N_u = 5$, $\lambda = 0.1$, $D_z = 10$



Rys. 1.26: Wykres wyjścia z pomiarem zakłócenia dla $D = 100$, $N = 20$, $N_u = 5$, $\lambda = 0.1$, $D_z = 10$

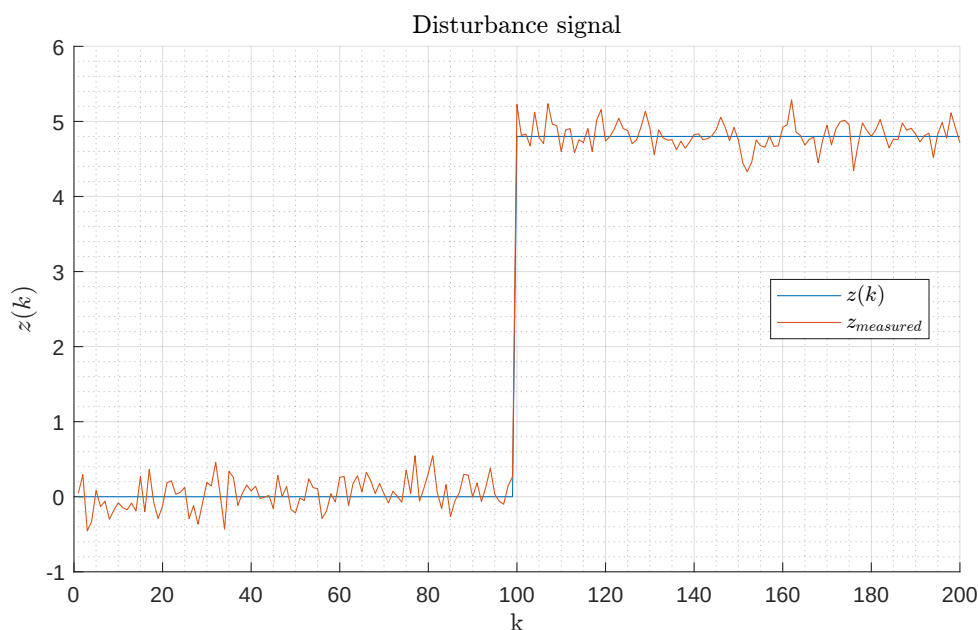
Jak widać na wykresach 1.23 - 1.26, pomiar zakłócenia dla zakłócenia sinusoidalnego nieznacznie wpływa na jakość regulacji. Wskaźniki jakości dla regulacji bez pomiaru jak i z pomiarem wyniosły odpowiednio: 8.292650, 8.284784.

1.2.4. Testy odporności regulatora DMC z pomiarem zakłócenia na szum pomiarowy

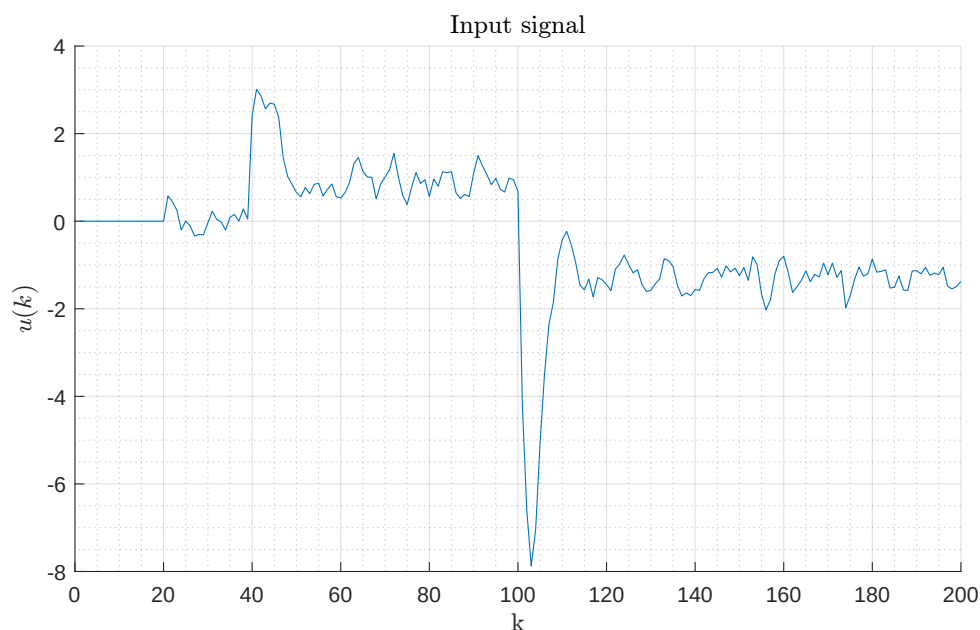
Zadanie te ma za zadanie zasymulować rzeczywisty obiekt, gdzie występuje zakłócenie mierzalne, dokonywany jest jego pomiar, ale niestety i na niego działają różne zakłócenia zewnętrzne i szumy pomiarowe, przez co informacja o zakłóceniu docierająca do regulatora jest nieznacznie różniąca się od rzeczywiście zmierzonego zakłócenia, którego wpływ ma niwelować regulator.

Wykres 1.27 przedstawia wartość rzeczywistego zakłócenia mierzalnego $z(k)$, natomiast z_{measured} jest sygnałem, który rzeczywiście trafia do algorytmu regulacji. W tym przypadku jest naniesiony szum o rozkładzie Gaussa o odchyleniu standardowym 0,2. Szum taki może wynikać na przykład z zakłóceń EM emitowanych przez jakiś nadajnik.

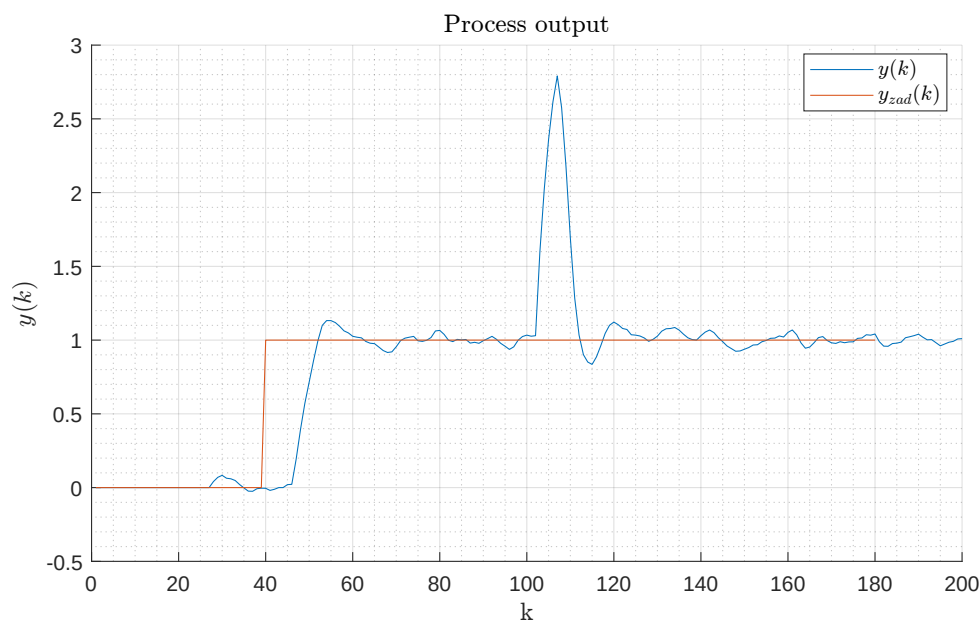
Wykresy 1.28 oraz 1.29 pokazują działanie regulatora w obecności takich szumów.



Rys. 1.27: Wykres skoku zakłócenia wraz z szumem Gaussowskim o odchyleniu standardowym 0.2

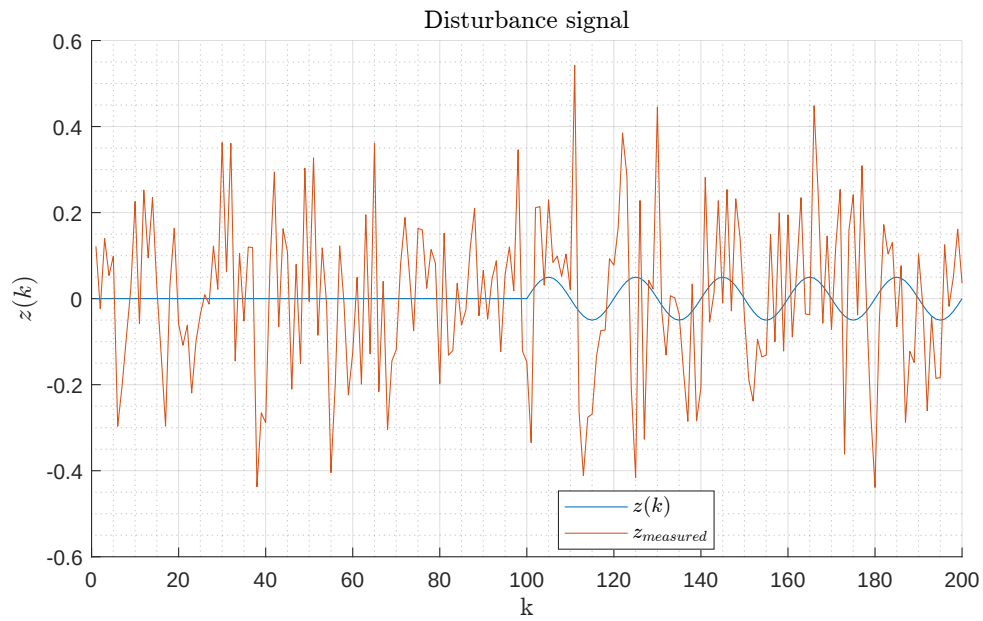


Rys. 1.28: Wykres sygnału wejściowego z pomiarem zakłócenia z szumem Gaussowskim dla $D = 100$, $N = 20$, $N_u = 5$, $\lambda = 0.1$, $D_z = 10$

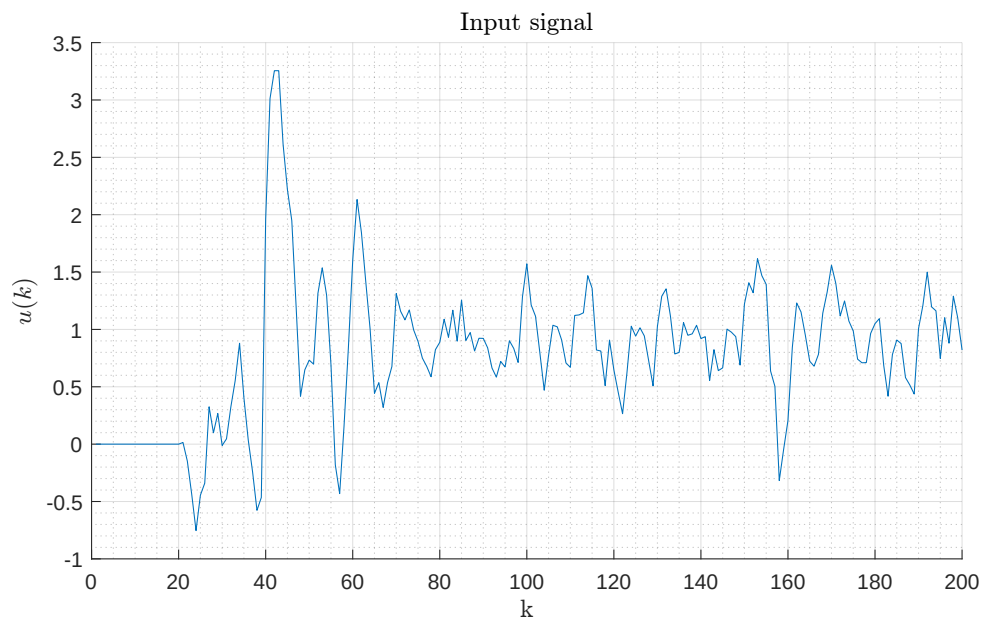


Rys. 1.29: Wykres wyjścia z pomiarem zakłócenia z szumem Gaussowskim dla $D = 100$, $N = 20$, $N_u = 5$, $\lambda = 0.1$, $D_z = 10$

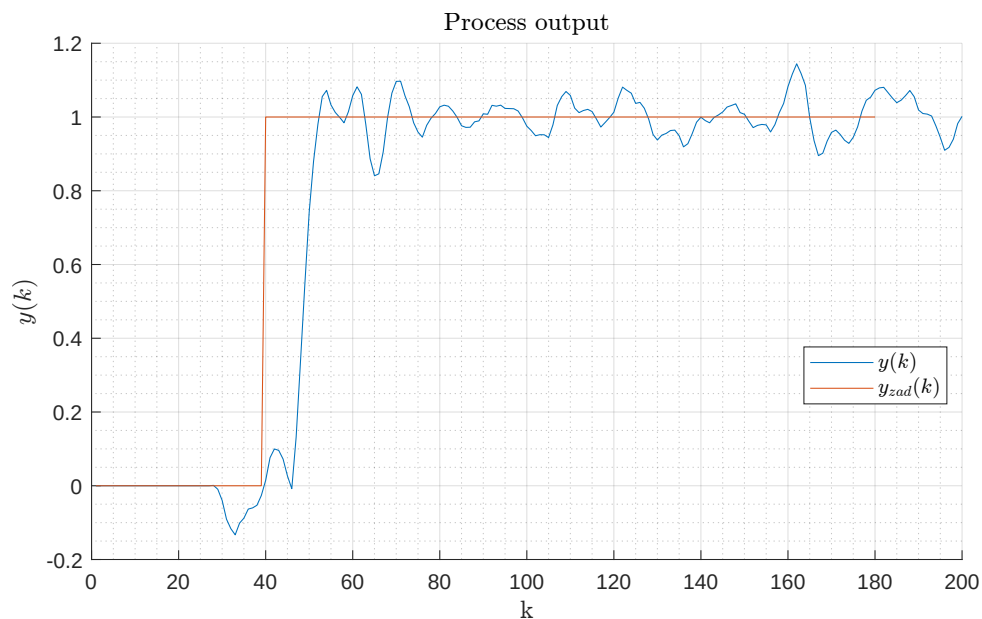
Wykres 1.30 przedstawia sygnał pomiarowy zakłócenia mierzalnego sinusoidalnego o amplitudzie 0,05 zaszumionego szumem Gaussa o amplitudzie 0,4, a więc skrajnie przewyższającym wartość sygnału użytecznego.



Rys. 1.30: Wykres skoku zakłócenia sinusoidalnego wraz z szumem Gaussowskim o odchyleniu standardowym 0.2



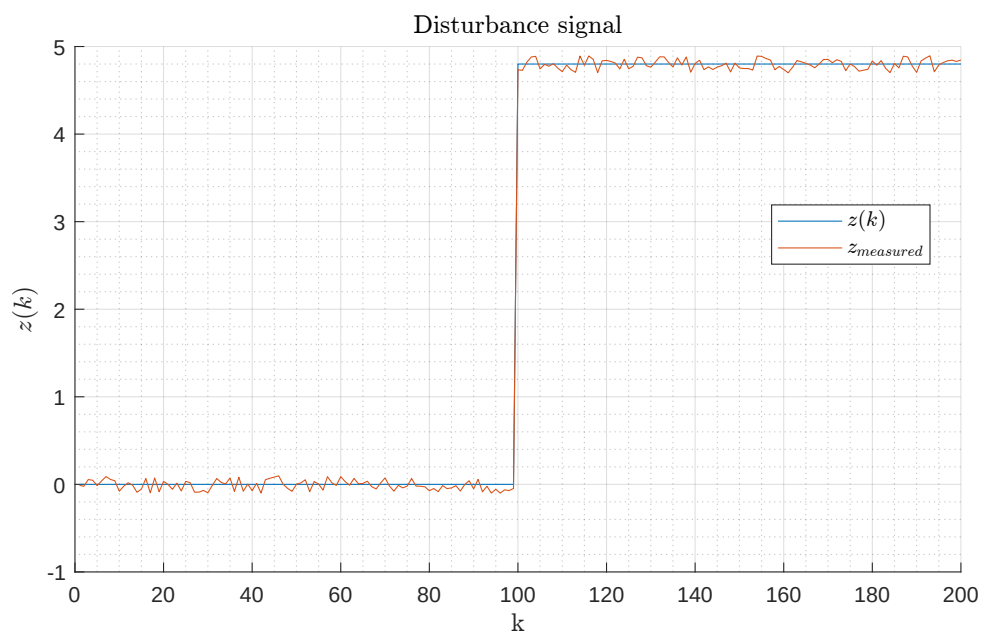
Rys. 1.31: Wykres sygnału wejściowego z pomiarem zakłócenia sinusoidalnego z szumem Gaussowskim dla $D = 100$, $N = 20$, $N_u = 5$, $\lambda = 0.1$, $D_z = 10$



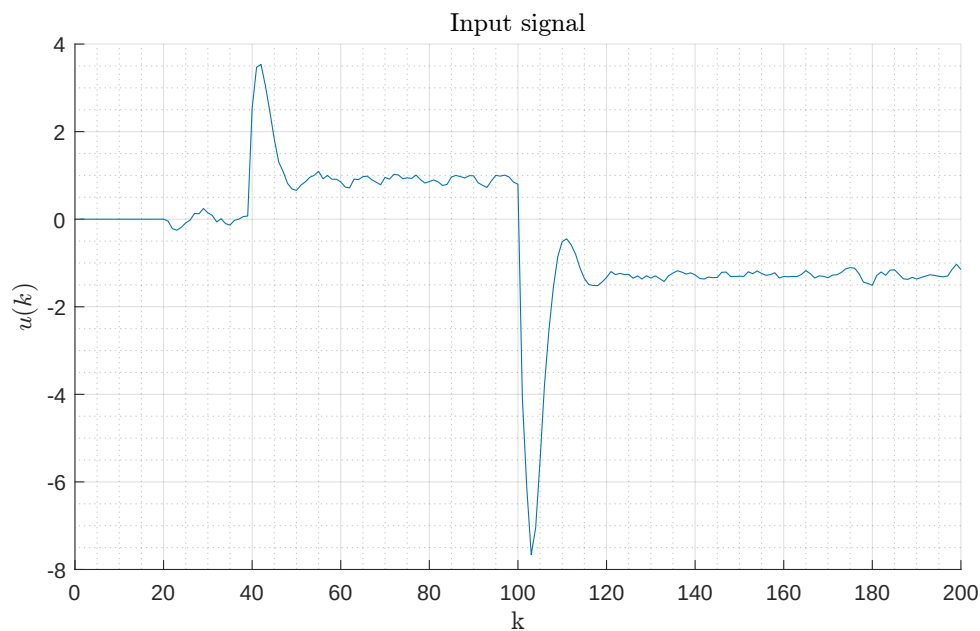
Rys. 1.32: Wykres wyjścia z pomiarem zakłócenia sinusoidalnego z szumem Gaussowskim dla $D = 100$, $N = 20$, $N_u = 5$, $\lambda = 0.1$, $D_z = 10$

Jak widać, mimo tak wielkiej amplitudy szumów względem użytecznego sygnału zakłóceń, sygnał wyjściowy $y(k)$ nie jest najgorszy.

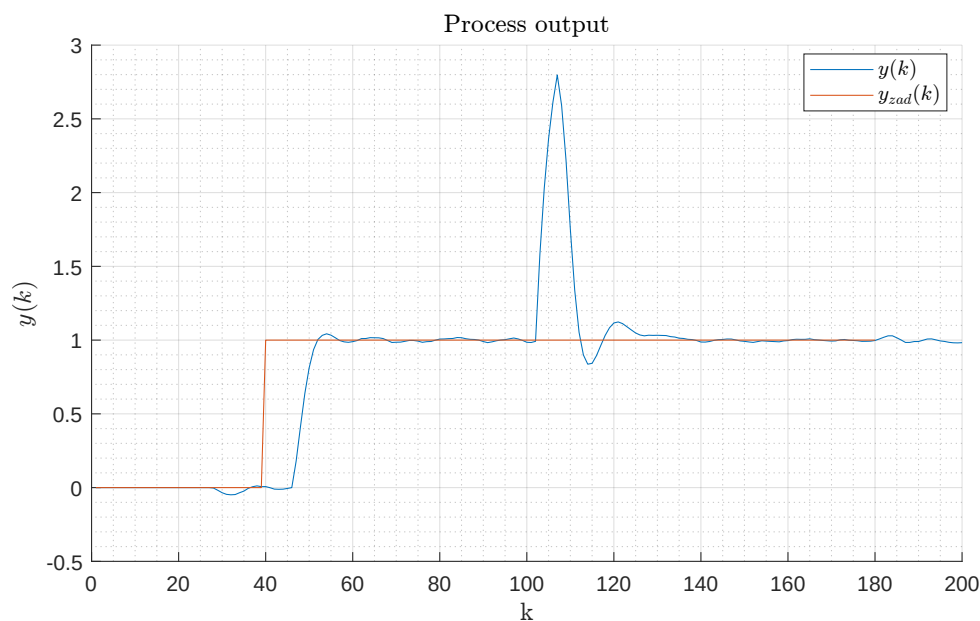
Na wykresie 1.33 przedstawiono skok zakłócenia z naniesionym szumem równomiernym o amplitudzie 0,1.



Rys. 1.33: Wykres skoku zakłócenia wraz z szumem równomiernym o amplitudzie 0,1

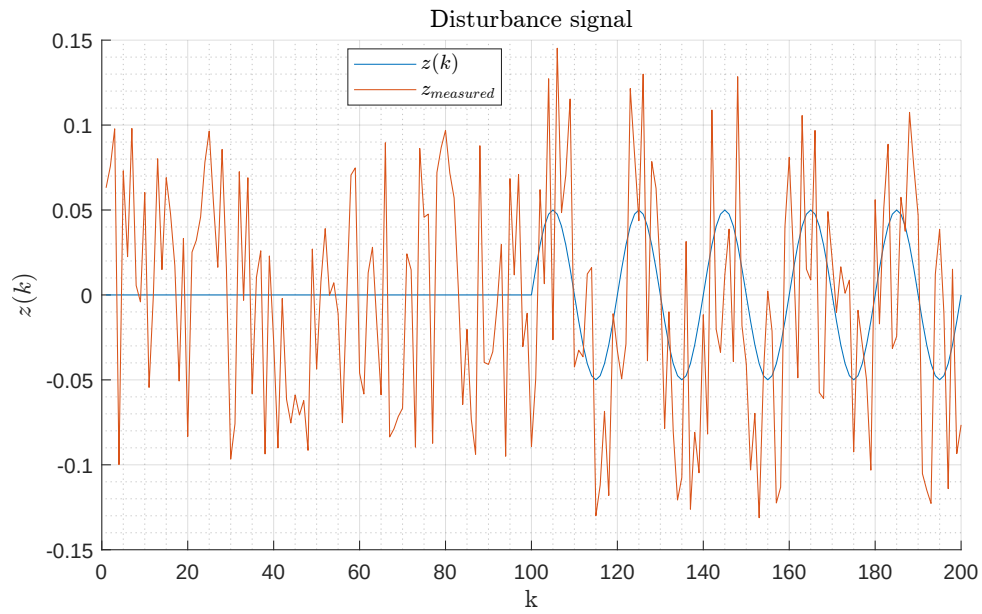


Rys. 1.34: Wykres sygnału wejściowego z pomiarem zakłócenia z szumem równomiernym dla $D = 100$, $N = 20$, $N_u = 5$, $\lambda = 0.1$, $D_z = 10$

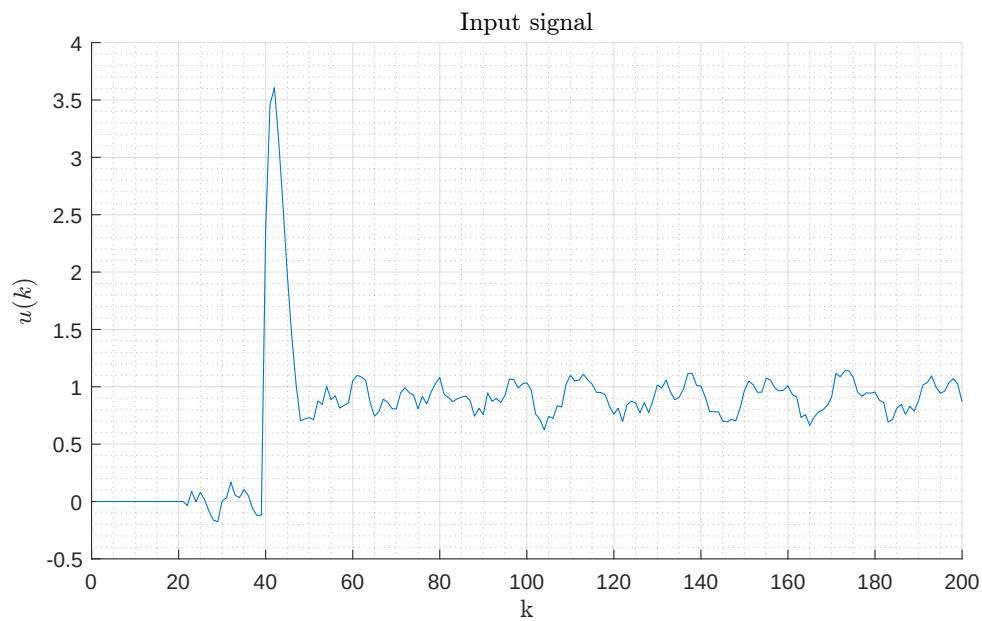


Rys. 1.35: Wykres wyjścia z pomiarem zakłócenia z szumem równomiernym dla $D = 100$, $N = 20$, $N_u = 5$, $\lambda = 0.1$, $D_z = 10$

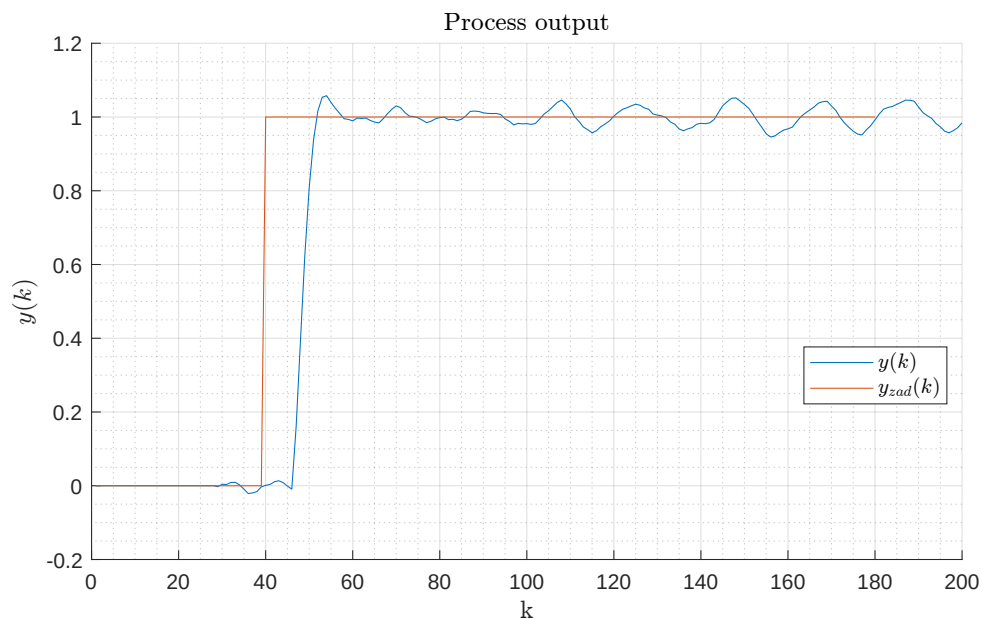
Na wykresie 1.36 przedstawiono skok zakłócenia sinusoidalnego z naniesionym szumem równomiernym o amplitudzie 0,1.



Rys. 1.36: Wykres skoku zakłócenia sinusoidalnego wraz z szumem równomiernym o amplitudzie 0.1

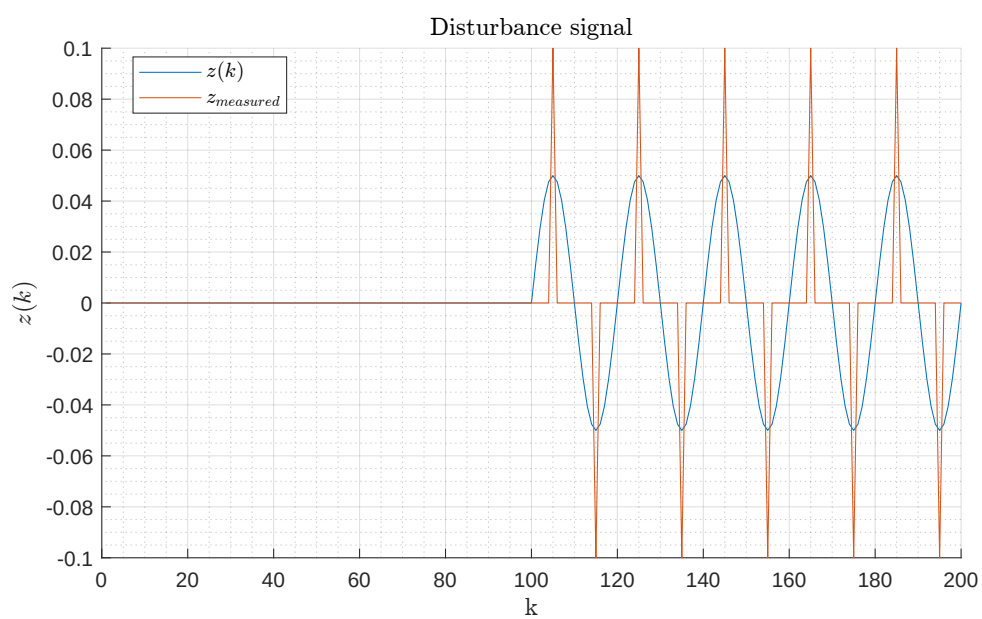


Rys. 1.37: Wykres sygnału wejściowego z pomiarem zakłócenia sinusoidalnego z szumem równomiernym dla $D = 100$, $N = 20$, $N_u = 5$, $\lambda = 0.1$, $D_z = 10$

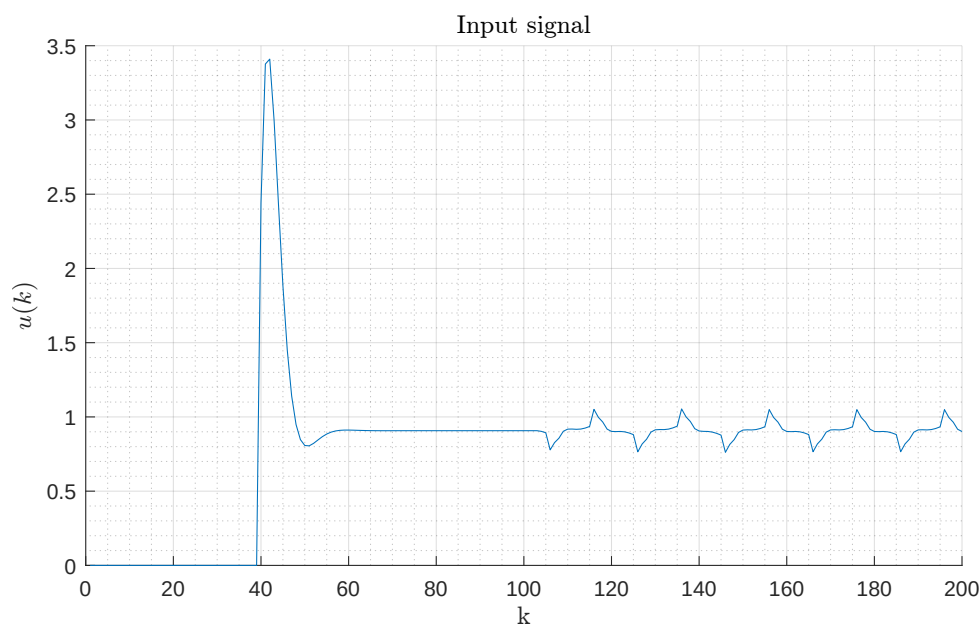


Rys. 1.38: Wykres wyjścia z pomiarem zakłócenia sinusoidalnego z szumem równomiernym dla $D = 100$, $N = 20$, $N_u = 5$, $\lambda = 0.1$, $D_z = 10$

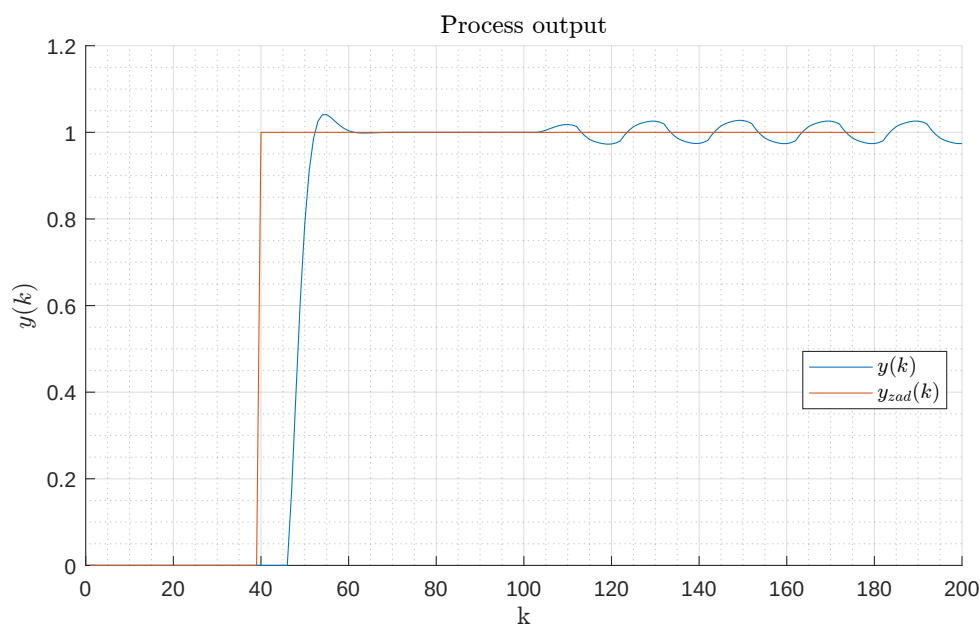
Na wykresie 1.39 przedstawiono skok zakłócenia sinusoidalnego z naniesionym szumem kwantyzacji $q = 0, 1$.



Rys. 1.39: Wykres skoku zakłócenia wraz z szumem kwantyzacji $q = 0.1$



Rys. 1.40: Wykres sygnału wejściowego z pomiarem zakłócenia sinusoidalnego z szumem kwantyzacji dla $D = 100$, $N = 20$, $N_u = 5$, $\lambda = 0.1$, $D_z = 10$



Rys. 1.41: Wykres wyjścia z pomiarem zakłócenia sinusoidalnego z szumem kwantyzacji dla $D = 100$, $N = 20$, $N_u = 5$, $\lambda = 0.1$, $D_z = 10$

Działanie algorytmu DMC z pomiarem zakłócenia przy wystąpieniu szumu pomiarowego nieznacznie się pogorszyło. Wskaźnik jakości wzrósł do następujących wartości:

Szum gaussowski: 23.060794, dla zakłócenia sinusoidalnego: 8.231972

Szum równomierny: 22.217082, dla zakłócenia sinusoidalnego: 8.134403

Szum kwantyzacji: 22.217582, dla zakłócenia sinusoidalnego: 8.283243

Wartości przeregulowania jak i tempo regulacji nie odbiegają znacząco od poprzednich przypadków. Regulator jest w stanie sobie poradzić z zakłóceniami jak i szumem pomiarowym.

2. Laboratorium

2.1. Identyfikacja procesu

2.1.1. Ustalanie punktu pracy stanowiska

Pracę w laboratorium rozpoczęto od przetestowania komunikacji ze stanowiskiem grzejąco-chłodzącym za pomocą skryptu `MinimalWorkingExample.m` dostępnego na stronie przedmiotu. Po stwierdzeniu poprawności działania systemu uruchomiono program `heating_station_set_work_point.m`, którego kod przedstawiono na listingu 2.1:

Listing 2.1: Skrypt do ustawienia punktu pracy

```
%% Script used in order to settle working point on the process output

Upp = 26;
FanPower = 50;
Z = 0;
kstart = 1;

%% Communication with heating station setup
addpath('D:\SerialCommunication');
initSerialControl COM7

y = zeros(100, 1);
environment = zeros(100, 1);

%% Configure plot
figure;
hold on;
h_y = stairs(1:kstart, y(1:kstart), 'DisplayName', 'y');
h_environment = stairs(1:kstart, environment(1:kstart), 'DisplayName', ...
    'environment');

title("Wykres sterowania i odpowiedzi stanowiska grzewczego");
xlabel("Iteracje [k]");
ylabel("Wartości");
legend;
grid on;

k = 1;
while(1)
    %% obtaining measurements
    measurements1 = readMeasurements(1); % T1 temperature
    measurements5 = readMeasurements(5); % T5 ambient temperature
    y(k) = measurements1;
    environment(k) = measurements5;

    fprintf('Heater temp: %.2f *C\tEnvironment temp: %.2f *C\n', ...
        measurements1, measurements5);

    %% sending new values of control signals
    sendControls(1, FanPower);
    sendControlsToG1AndDisturbance(Upp, 0);
end
```

```

% Update plots
set(h_y, 'XData', 1:k, 'YData', y(1:k));
set(h_environment, 'XData', 1:k, 'YData', environment(1:k));
drawnow;
k = k + 1;

%% synchronising with the control process
waitForNewIteration();
end

```

Skrypt ten ustawia sterowanie U_{pp} na wartość wyznaczoną dla naszego numeru zespołu, czyli na $25 + \text{mod } 11 = 26$. Dodatkowo program ustawia prędkość wentylatora na 50%, a także rysuje wykresy temperatury obiektu T1 oraz temperatury otoczenia oraz wypisuje te dane w terminalu, co pozwala łatwo zobaczyć, czy stanowisko osiągnęło właściwy punkt pracy.

2.1.2. Zbieranie odpowiedzi skokowych dla toru zakłócenie → wyjście procesu

Po osiągnięciu stanu ustalonego przez obiekt, za pomocą skryptu 2.2 - `saving_disturbance_step_response.m` wykonywano skok wartości zakłócenia, a następnie zbierano kolejne odczyty z wyjścia obiektu w celu późniejszej aproksymacji transmitancji toru zakłócenie → wyjście obiektu.

Listing 2.2: Skrypt do zbierania odpowiedzi skokowej toru zakłócenie → wyjście obiektu

```

%% Communication with heating station setup
addpath('D:\SerialCommunication');
initSerialControl COM7

%% Experiment parameters
N = 1000;
Upp = 26;
FanPower = 50;
step_value_1 = 5; % z(k) value after step
T1 = zeros(N, 1);

%% Data export setup
name = "data/zad2_disturbance2_step=" + string(step_value_1) + ".csv";
file_id = fopen(name, 'a');
fprintf(file_id, "heater_temp, environment_temp\n");
buffer_size = 15;
buffer = zeros(buffer_size, 2);
buffer_index = 1;
kstart = 1;
y = zeros(N, 1);

h_y = stairs(1:kstart, y(1:kstart), 'DisplayName', 'y');

title("Wykres odpowiedzi toru z-> y stanowiska grzewczego");
xlabel("Iteracje [k]");
ylabel("Wartości");
legend;
grid on;

%% Make the jump and change to the sendControlToG1AndDisturbance()
sendControls(1, FanPower);
sendControlsToG1AndDisturbance(Upp, step_value_1);

for k=1:N
    measurements = readMeasurements([1 5]);
    fprintf('Heater temp: %.2f *C\tEnvironment temp: %.2f *C\n', ...
            measurements(1), measurements(2));

    T1(k) = measurements(1);

```

```
y(k) = T1(k);

set(h_y, 'XData', 1:k, 'YData', y(1:k));
drawnow;

% saving to buffer
buffer(buffer_index, :) = measurements;
buffer_index = buffer_index + 1;

if buffer_index > buffer_size
    fprintf(file_id, '%f, %f\n', buffer');
    buffer_index = 1;
end

waitForNewIteration();

end

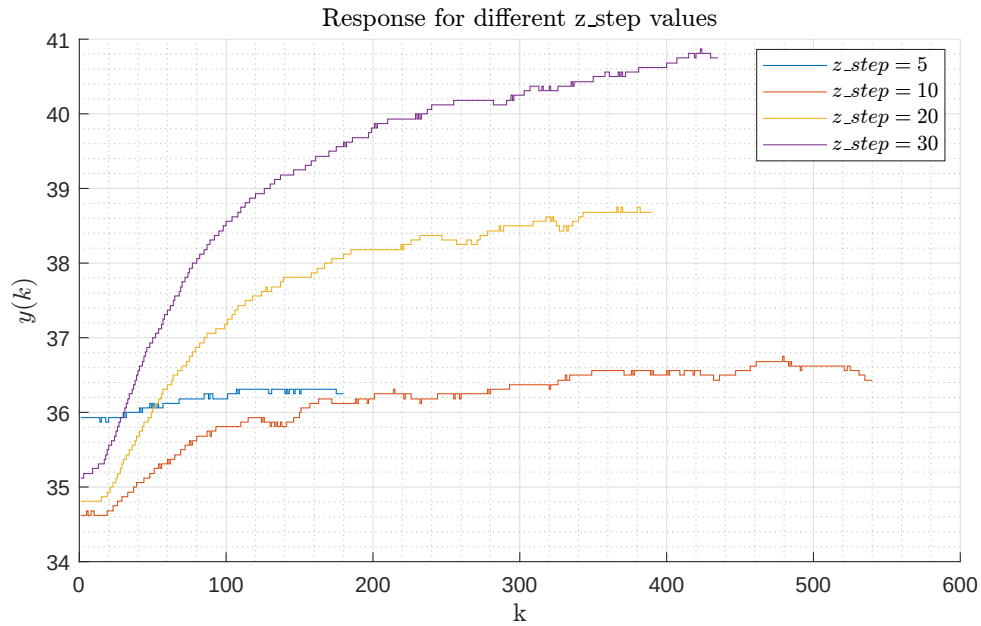
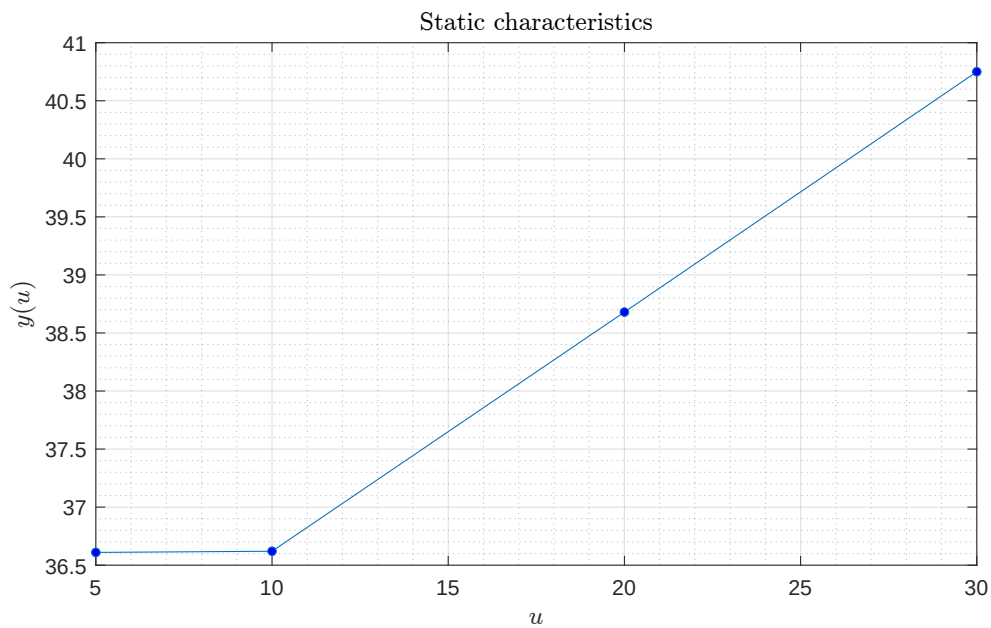
% saving buffer if it isn't empty
if buffer_index > 1
    fprintf(file_id, '%f, %f\n', buffer(1:buffer_index-1, :));
end
fclose(file_id);

%% Send working point parameters
sendControls(1, FanPower);
sendControlsToG1AndDisturbance(Upp, 0);
```

Warto podkreślić, że w tym ćwiczeniu laboratoryjnym zakłóceniem mierzalnym nie był żaden sygnał fizyczny, a zakłócenie było jedynie symulowane za pomocą funkcji `sendControlsToG1AndDisturbance()` dostarczonej przez Prowadzącego. Powyższy skrypt 2.2, podobnie jak poprzedni, rysuje na bieżąco wykres temperatury oraz dodatkowo zapisuje dane do bufora, który jest cyklicznie zapisywany do pliku.

Jako że w poleceniu podano, że testowane mają być skoki zakłócenia od 0 do 30, eksperyment powtórzono dla wartości skoków 5, 10, 20, 30, oczywiście za każdym razem wykonując skrypt 2.1 - `heating_station_set_work_point.m` i czekając, aż obiekt powróci do stanu ustalonego.

W ten sposób zebrano dane, które przedstawiono na wykresie 2.1. Każdy pomiar był innej długości, gdyż dla różnych skoków różny czas był potrzebny, aby móc stwierdzić, że stan się już ustalił.

Rys. 2.1: Odpowiedzi skokowe dla toru $z \rightarrow y$ zebrane na stanowisku grzejąco-chłodzącymRys. 2.2: Charakterystyka statyczna toru $z \rightarrow y$ stanowiska grzejąco-chłodzącego

$$K_{stat\ z \rightarrow y} = \frac{1.05}{5} = 0.21 \quad (2.1)$$

Na podstawie zebranych danych narysowano charakterystykę statyczną toru $z \rightarrow y$, która została przedstawiona na wykresie 2.2. Na pierwszy rzut oka może wydawać się ona dziwna, gdyż na początku jest płaska, a następnie rośnie liniowo. Jednak wytłumaczenie tego jest bardzo proste. Skoki wartości zakłócenia z były wykonywane od zera do wartości po kolei 10, 20, 30 i na koniec wykonano dodatkowy skok dla wartości 5. Jak widać na wykresie 2.1, im później był wykonywany eksperyment, tym punkt początkowy y był coraz wyżej, mimo że zawsze czekano bardzo długo na powrót temperatury obiektu do punktu początkowego. Wynika to ze wzrostu

temperatury w pracowni podczas zajęć. Dla każdego wykonywanego skoku, zbierano dane także i o temperaturze otoczenia, dzięki temu wiadomo, że dla skoku pierwszego (`step_value = 10`) temperatura otoczenia wynosiła $27,93^{\circ}\text{C}$, natomiast dla ostatniego eksperymentu (`step_value = 5`) temperatura w laboratorium wyniosła już $29,81^{\circ}\text{C}$. Co prawda tak wysokie temperatury wydają się nieprawdopodobne, przy 29°C bardzo ciężko jest już funkcjonować i temperatury takie występują tylko latem. Jednak jest jedno potencjalne źródło takiego zawyżania temperatury. Zauważono, że czujnik temperatury otoczenia zamontowany na luźnym przewodzie to bardzo popularny DS16B20, który jest cyfrowym czujnikiem komunikującym się po magistrali OneWire. Według noty katalogowej ma on tolerancję $\pm 0,5^{\circ}\text{C}$, jednak bardzo istotne jest w nim to, że każdy pomiar powoduje wydzielenie się na nim bardzo małej mocy, która może powodować nieznaczne podwyższenie się temperatury struktury półprzewodnikowej i zafałszowanie pomiaru w przypadku, gdy czujnik jest zbyt często odczytywany. Podejrzewa się, że to może powodować tak zawyżone odczyty w tym przypadku, gdyż co prawda komunikacja z komputerem jest dokonywana co około 1 s, to w rzeczywistości komunikacja z czujnikami temperatury może być przeprowadzana znacznie częściej. Jednak takie zawyżenie pomiaru powinno być mniej więcej stałe w przypadku stanu równowagi termodynamicznej czujnika, a więc mimo wszystko pokazuje on prawidłowo tendencję do wzrostu temperatury w pracowni, co było także odczuwalne organoleptycznie, a co przekłada się niestety na jakość zebranych modeli i późniejszą jakość regulacji algorytmem DMC, który jest przecież bardzo podatny na zmiany modelu.

Tak więc można śmiało przyjąć, że charakterystyka statyczna toru zakłóceń jest liniowa.

2.1.3. Aproksymacja odpowiedzi skokowych dla toru zakłócenie $\rightarrow$ wyjście procesu za pomocą członu inercyjnego drugiego rzędu z opóźnieniem

Jako że na rzeczywisty obiekt działają także różne zakłócenia niemierzalne oraz nieuniknione jest też szum pomiarowy, nie można bezpośrednio użyć w algorytmie DMC odpowiedzi zebranych wcześniej. Należy najpierw zaproksymować dane za pomocą jakiejś funkcji, co pozwoli w znaczącym stopniu ograniczyć wpływ zakłóceń, które w ogólności są składowymi o wyższych częstotliwościach widmowych niż badany sygnał. Do aproksymacji użyto model dwuinercyjny z opóźnieniem.

Analityczna postać aproksymowanej transmitancji

Zebrane odpowiedzi skokowe posłużyły do aproksymacji transmitancji toru zakłócenie $\rightarrow$ wyjście obiektu jako człon dwuinercyjny z opóźnieniem, czyli tak samo jak na laboratorium 1. był aproksymowany tor wejście $\rightarrow$ wyjście obiektu. Dla przypomnienia podano wzory:

$$G^z(s) = \frac{K^z}{(sT_1^z + 1)(sT_2^z + 1)} e^{-T_d^z T_p^z s} \quad (2.2)$$

gdzie T_p^z to czas próbkowania, w tym zadaniu równy 1 s. Jak widać, jedyną różnicą jest zastosowany indeks z podkreślający, że chodzi o transmitancję toru zakłóceń. W dziedzinie czasu dyskretnego transmitancja ta przyjmuje postać:

$$G^z(z) = \frac{b_1^z z^{-1} + b_2^z z^{-2}}{1 + a_1^z z^{-1} + a_2^z z^{-2}} z^{-T_d^z} \quad (2.3)$$

w której zastosowane współczynniki są równe:

$$\begin{aligned}
a_1^z &= -\alpha_1^z - \alpha_2^z \\
a_2^z &= \alpha_1^z \alpha_2^z \\
\alpha_1^z &= e^{-\frac{1}{T_1^z}} \\
\alpha_2^z &= e^{-\frac{1}{T_2^z}} \\
b_1^z &= \frac{K^z}{T_1^z - T_2^z} [T_1^z(1 - \alpha_1^z) - T_2^z(1 - \alpha_2^z)] \\
b_2^z &= \frac{K^z}{T_1^z - T_2^z} [\alpha_1^z T_2^z(1 - \alpha_2^z) - \alpha_2^z T_1^z(1 - \alpha_1^z)]
\end{aligned} \tag{2.4}$$

Na podstawie transmitancji dyskretnej otrzymuje się następujące równanie różnicowe:

$$y^z(k) = b_1^z z(k - T_d^z - 1) + b_2^z z(k - T_d^z - 2) - a_1^z y(k - 1) - a_2^z y(k - 2) \tag{2.5}$$

Do aproksymacji wykorzystano ten sam algorytm co na laboratorium 1., dlatego też w poniższych listingach, które stanowią przypomnienie tego algorytmu, nie stosowano w oznaczeniach indeksów ^z, gdyż powyżej użyto ich tylko do pokreślenia, że jest to transmitancja toru zakłócenie → wyjście obiektu.

Implementacja algorytmu aproksymacji

Na początku zaimplementowano funkcję `heating_station_simulation()`, która służy do obliczenia wartości równania różnicowego 2.5 i która będzie wykorzystywana w kolejnych funkcjach:

Listing 2.3: Funkcja `heating_station_simulation()`

```
function [y] = heating_station_simulation(uktdm1, uktdm2, ykm1, ykm2, a, b)

y = b(1)*uktdm1 + b(2)*uktdm2 - a(1)*ykm1 - a(2)*ykm2;

end
```

Powyższa funkcja wymaga podania wektorów współczynników a i b , za których obliczenie na podstawie K , T_1 i T_2 odpowiada funkcja `calculate_coefficients()`:

Listing 2.4: Funkcja `calculate_coefficients()`

```
function [a, b] = calculate_coefficients(T1, T2, K)

alpha_1 = exp(-1/T1);
alpha_2 = exp(-1/T2);

a = zeros(2, 1);
b = zeros(2, 1);

a(1) = - alpha_1 - alpha_2;
a(2) = alpha_1*alpha_2;

b(1) = K/(T1 - T2)*(T1*(1 - alpha_1) - T2*(1 - alpha_2));
b(2) = K/(T1 - T2)*(alpha_1*T2*(1 - alpha_2) - alpha_2*T1*(1 - alpha_1));

end
```

Proces aproksymacji polega na znalezieniu takich wartości współczynników transmitancji K , T_1 oraz T_2 , aby przebieg funkcji aproksymującej jak najbardziej przypominał zebrane dane. Aby móc przeprowadzić ten proces, potrzebna jest funkcja celu - miara jakości aproksymacji. W tym

przypadku zastosowano sumę kwadratu błędów dla każdej próbki. Zostało to zaimplementowane w MATLAB jako funkcja `approx_goal_func()`:

Listing 2.5: Funkcja `approx_goal_func()`

```
function [error] = approx_goal_func(td, Upp, step_value, y_measured, K, T1, T2)
    [a, b] = calculate_coefficients(T1, T2, K);

    N = length(y_measured);
    u = ones(N, 1) * step_value;
    y = ones(N, 1) * y_measured(1);

    error = zeros(N, 1);
    for k=1:N
        if(k - td - 1 < 1)
            uktdm1 = Upp;
        else
            uktdm1 = u(k - td - 1);
        end

        if(k - td - 2 < 1)
            uktdm2 = Upp;
        else
            uktdm2 = u(k - td - 2);
        end

        if(k - 1 < 1)
            ykm1 = y_measured(1);
        else
            ykm1 = y(k-1);
        end

        if(k - 2 < 1)
            ykm2 = y_measured(1);
        else
            ykm2 = y(k-2);
        end

        y(k) = heating_station_simulation(uktdm1, uktdm2, ykm1, ykm2, a, b);
        error(k) = (y(k) - y_measured(k))^2;
    end
    error = sum(error);
end
```

W powyższym skrypcie użyto instrukcje warunkowe `if`, aby podstawić do wzoru odpowiednie wartości u oraz y , gdy wartości indeksów w równaniu różnicowym byłyby niedodatnie. Inną metodą byłoby odpowiednie przesunięcie indeksów, po których iteruje pętla `for`. Jednak celowo wybrano te podejście, gdyż czyni kod bardziej intuicyjnym i pozwala łatwo użyć go także do innych celów.

Wszystkie powyżej przedstawione funkcje są wykorzystywane przez główną funkcję do aproksymacji - `approximation()`. Przyjmuje ona jako argumenty wartość skoku, punkt pracy oraz dane z pliku CSV. Następnie dokonuje normalizacji danych, gdyż jest to bardzo ważne, aby funkcja wyznaczała jakość aproksymacji zawsze zwracała wiarygodny wynik - dla transmitancji z opóźnieniem lub bez, dla skoków o dużej lub małej wartości. Następnie z użyciem funkcji `fmincon()` są wyznaczane najlepsze parametry K , T_1 , T_2 dla kolejnych wartości opóźnienia T_d od 1 do 15. Po czym wybierana jest wartość opóźnienia, dla której uzyskało się najlepszą jakość aproksymacji i w ten sposób uzyskuje się poprawny i optymalny zestaw współczynników. Gdyby w funkcji `fmincon()` optymalizować bezpośrednio wszystkie 4 współczynniki, to algorytm mógłby w błędny sposób starać się zmieniać wpływ opóźnienia T_d .

Listing 2.6: Funkcja approximation()

```

function [best_params, best_td] = approximation(step_value, ...
    working_point, raw_data)

    output = raw_data(:, 1);

    % Initial values of the searched transfer function
    K = 50;
    T1 = 2;
    T2 = 3;

    output = (output - ones(size(output))* ...
        output(1))/(step_value - working_point);

    best_error = inf;
    best_params = [];
    best_td = 0;

    for td = 1:15
        x_pocz = [K, T1, T2];
        upper_constraint = [150 100 100];
        bottom_constraint = [0.1 1.2 1];

        xopt = fmincon(@(x_init) approx_goal_func(td, 0, 1, ...
            output, x_init(1), x_init(2), x_init(3)), ...
            x_pocz, [], [], [], [], ...
            bottom_constraint, upper_constraint);

        error = approx_goal_func(td, 0, 1, ...
            output, xopt(1), xopt(2), xopt(3));

        if(error <= best_error)
            best_error = error;
            best_params = xopt;
            best_td = td;
        end
    end
end

```

2.1.4. Otrzymywanie wektora odpowiedzi skokowej dla algorytmu DMC

Do działania algorytmu DMC potrzebny jest wektor współczynników odpowiedzi skokowej s , który przedstawia kolejne wartości wyjścia obiektu w kolejnych kwantach czasu po wykonaniu skoku jednostkowego na wejściu. Jako że do aproksymacji używano znormalizowanych danych, teraz już nie trzeba wykonywać tej operacji. Dlatego napisana funkcja `step_response()` jest bardzo prosta. Przyjmuje ona wartości U_{PP} oraz Y_{PP} w punkcie pracy, wartość horyzontu dynamiki oraz współczynniki członu dwuinercyjnego z opóźnieniem.

Listing 2.7: Funkcja step_response()

```

function y = step_response(Upp, Ypp, D, K, T1, T2, td)
    u = ones(D, 1) * 1;
    y = ones(D, 1) * 0;
    [a, b] = calculate_coefficients(T1, T2, K);

    for k=2:D
        if(k - td - 1 < 1)
            uktdm1 = Upp;
        else
            uktdm1 = u(k - td - 1);
        end
    end
end

```

```

    if(k - td - 2 < 1)
        uktdm2 = Upp;
    else
        uktdm2 = u(k - td - 2);
    end

    if(k - 1 < 1)
        ykm1 = Ypp;
    else
        ykm1 = y(k-1);
    end

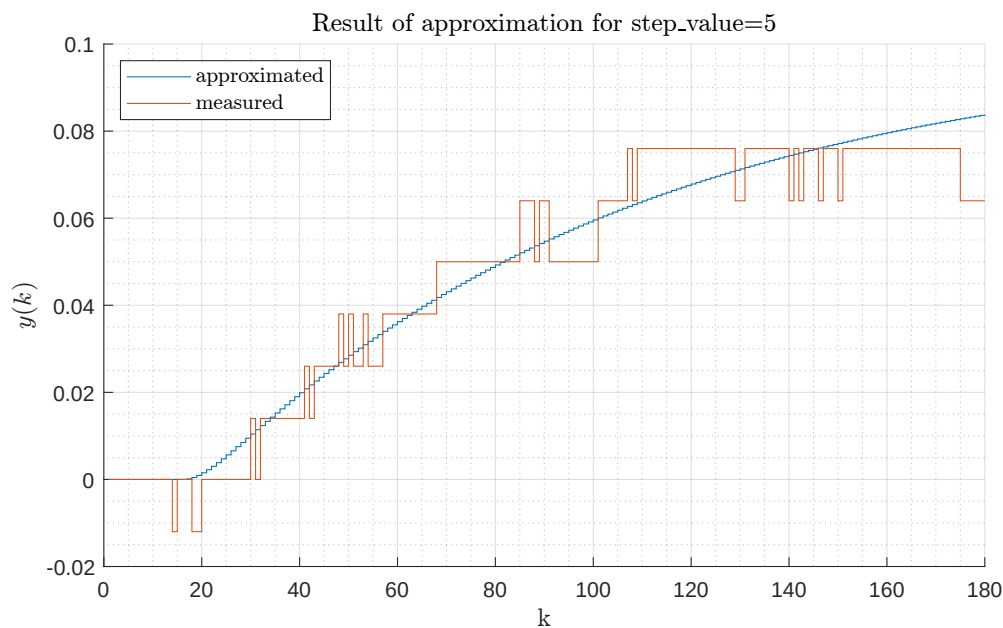
    if(k - 2 < 1)
        ykm2 = Ypp;
    else
        ykm2 = y(k-2);
    end

    y(k) = heating_station_simulation(uktdm1, uktdm2, ykm1, ykm2, a, b);
end
end

```

W celu przetestowania algorytmu aproksymacji oraz otrzymywania znormalizowanego wektora odpowiedzi skokowej s , pobierano dane z odpowiednich plików CSV, dokonywano aproksymacji, a następnie wywoływano funkcję `step_response()` z ustawioną wartością horyzontu dynamiki D_z równą długości danych z pliku CSV, po czym rysowano wykresy, które przedstawiono poniżej.

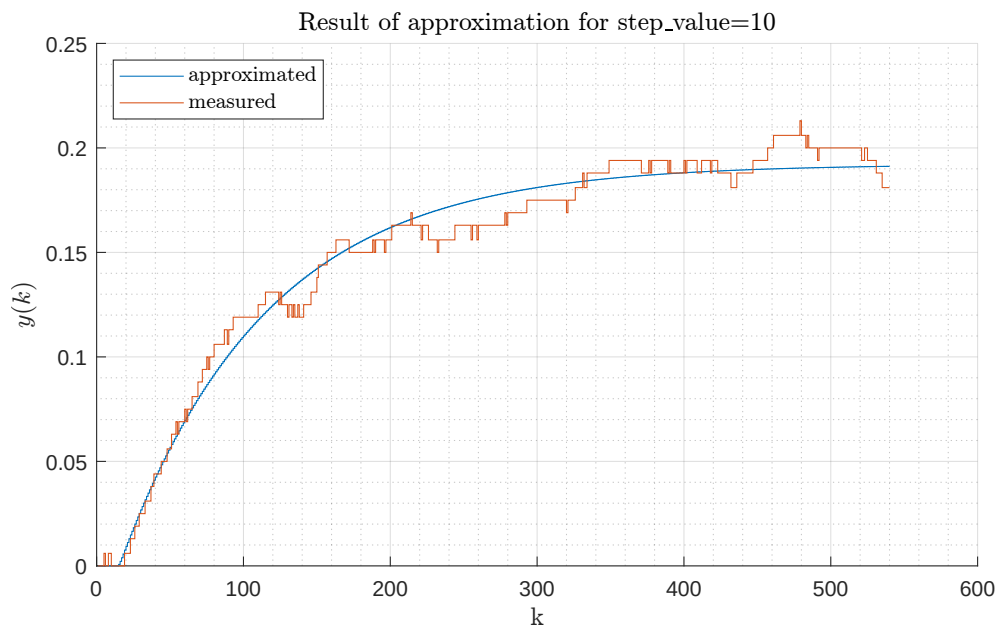
Na rysunku 2.3 został przedstawiony wykres otrzymanej zaproksymowanej odpowiedzi skokowej dla skoku zakłóceń równego 5. Można zauważyć na nim, że jakość aproksymacji jest w tym przypadku słaba - wartość na wyjściu modelu rosłaby nadal.



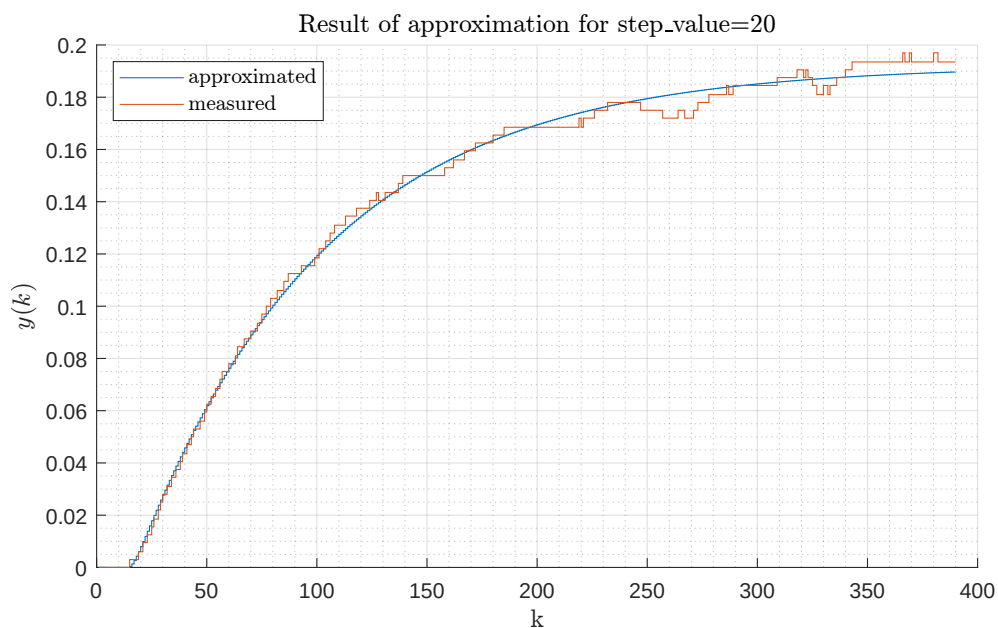
Rys. 2.3: Porównanie znormalizowanych odpowiedzi skokowych toru $z \rightarrow y$ zebranych na stanowisku grzejąco-chłodzącym oraz otrzymanych w wyniku aproksymacji dla `step_value=5`

Wynika to z tego, że, jak widać na wykresie 2.1, dla tego niewielkiego skoku zakłóceń wartość

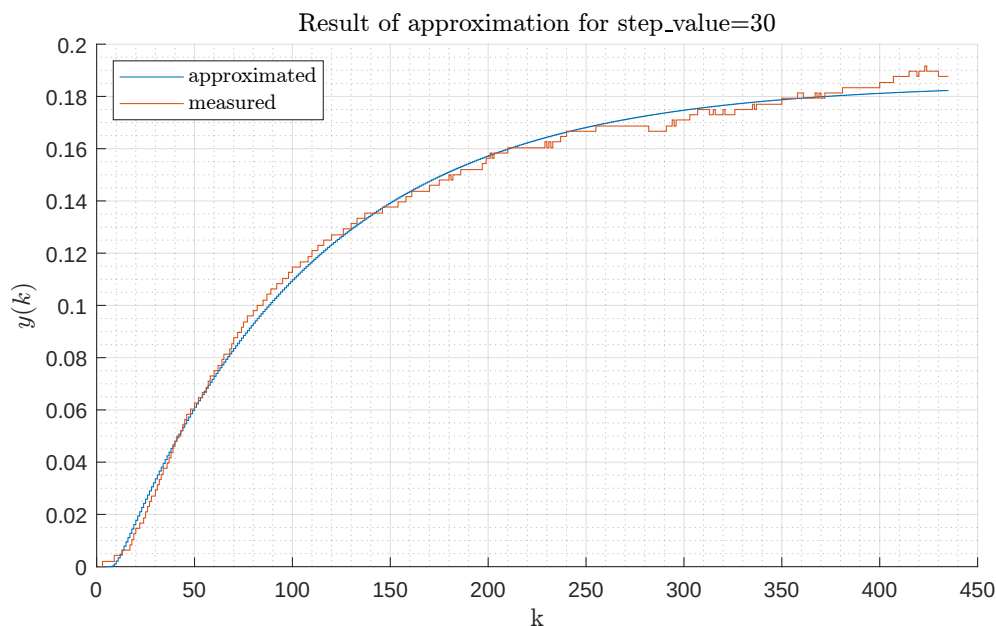
y się zmieniła o jedynie kilka dziesiątych stopnia, co ginie w zakłóceniach niemierzalnych i szumie pomiarowym. Przedstawia się to na wykresie w postaci dużej amplitudy oscylacji wartości zmierzonej na powyższym wykresie.



Rys. 2.4: Porównanie znormalizowanych odpowiedzi skokowych toru $z \rightarrow y$ zebranych na stanowisku grzejąco-chłodzącym oraz otrzymanych w wyniku aproksymacji dla **step_value=10**



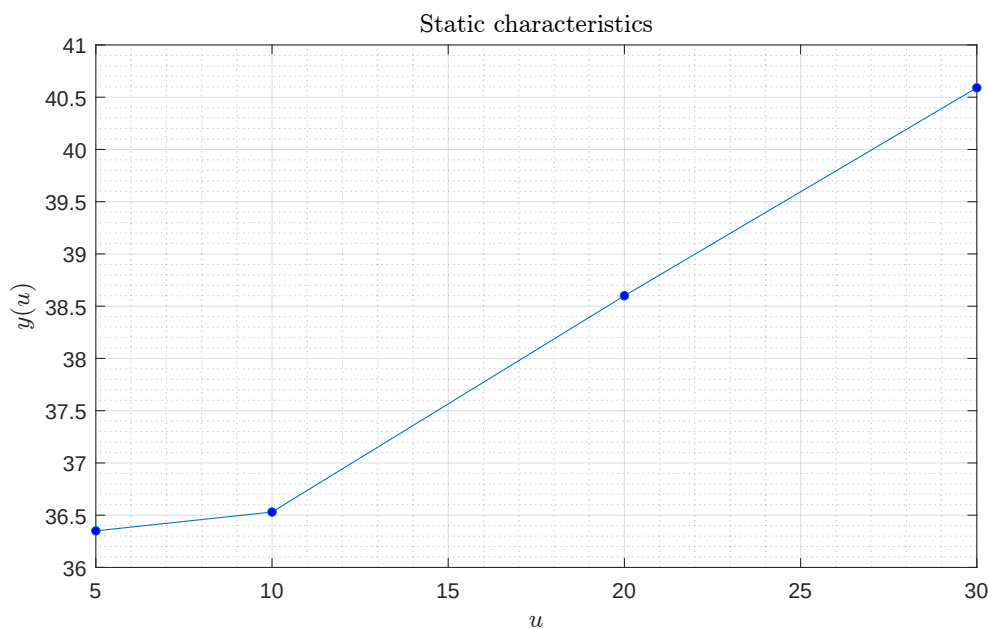
Rys. 2.5: Porównanie znormalizowanych odpowiedzi skokowych toru $z \rightarrow y$ zebranych na stanowisku grzejąco-chłodzącym oraz otrzymanych w wyniku aproksymacji dla **step_value=20**



Rys. 2.6: Porównanie znormalizowanych odpowiedzi skokowych toru $z \rightarrow y$ zebranych na stanowisku grzejąco-chłodzącym oraz otrzymanych w wyniku aproksymacji dla `step_value=30`

Na wykresach 2.4, 2.5, 2.6 widać zmierzone i zaproksymowane odpowiedzi skokowe dla kolejnych wartości `step_value`. Im większa wartość skoku z , tym większa zmiana wartości y , a więc mniejszy wpływ innych zakłóceń zewnętrznych. Dlatego też im większa wartość skoku, tym lepsza jakość aproksymacji.

Narysowano ponownie charakterystykę statyczną, tym razem wykorzystując otrzymane wcześniej zaproksymowane odpowiedzi skokowe. Na wykresie 2.7 da się zobaczyć nieznaczną poprawę względem charakterystyki otrzymanej wcześniej (2.2).



Rys. 2.7: Charakterystyka statyczna toru $z \rightarrow y$ stanowiska grzejąco-chłodzącego na podstawie zaproksymowanych odpowiedzi skokowych

2.1.5. Zbieranie odpowiedzi skokowych dla toru wejście → wyjście procesu oraz aproksymacja za pomocą członu inercyjnego drugiego rzędu z opóźnieniem

W tym bloku laboratoryjnym było używane te samo stanowisko, co na laboratorium 1., a więc można by było zastosować ten sam model odpowiedzi skokowej, który wyznaczono wcześniej. Jednak tym razem po wejściu do laboratorium dało się odczuć, że temperatura otoczenia była niższa niż poprzednim razem, a więc w celu otrzymania jak najlepszego działania algorytmu DMC, powtórzono identyfikację modelu dla toru wejście → wyjście obiektu, wykonując skok u o wartości 45. Do tego celu posłużył skrypt `saving_control_step_response.m` przedstawiony poniżej. Do aproksymacji użyto tego samego algorytmu opisanego wcześniej.

Listing 2.8: Skrypt do zbierania odpowiedzi skokowej toru wejście → wyjście obiektu

```
% Communication with heating station setup
addpath('D:\SerialCommunication');
initSerialControl COM7

%% Experiment parameters
N = 1000;
Upp = 26;
FanPower = 50;
step_value_1 = 1; % u(k) value after step
T1 = zeros(N, 1);

%% Data export setup
name = "data/zad2_control_step=" + string(step_value_1) + ".csv";
file_id = fopen(name, 'a');
fprintf(file_id, "heater_temp, environment_temp\n");
buffer_size = 15;
buffer = zeros(buffer_size, 2);
buffer_index = 1;

%% Make the jump
sendControls([1, 5], [FanPower, step_value_1]);

for k=1:N
    measurements = readMeasurements([1 5]);
    fprintf('Heater temp: %.2f *C\tEnvironment temp: %.2f *C\n', ...
        measurements(1), measurements(2));

    T1(k) = measurements(1);
    stairs(1:k, T1(1:k));
    drawnow;

    % saving to buffer
    buffer(buffer_index, :) = measurements;
    buffer_index = buffer_index + 1;

    if buffer_index > buffer_size
        fprintf(file_id, '%f, %f\n', buffer');
        buffer_index = 1;
    end

    waitForNewIteration();
end

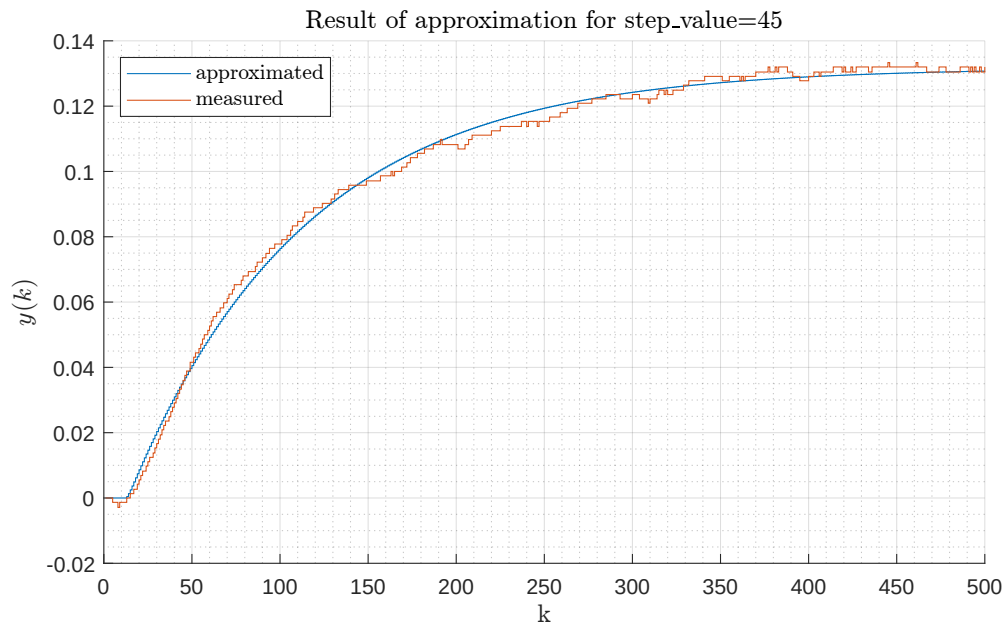
% saving buffer if it isn't empty
if buffer_index > 1
    fprintf(file_id, '%f, %f\n', buffer(1:buffer_index-1, :));
end
```



```
fclose(file_id);

%% Send working point parameters
sendControls([1, 5], [FanPower, Upp]);
```

Znormalizowana i zaproksymowana odpowiedź skokowa s dla wartości skoku 45 ma się następująco:



Rys. 2.8: Porównanie znormalizowanych odpowiedzi skokowych toru $u \rightarrow y$ zebranych na stanowisku grzejąco-chłodzącym oraz otrzymanych w wyniku aproksymacji dla `step-value=45`

2.2. Analityczny algorytm DMC z pomiarem zakłócenia

2.2.1. Implementacja algorytmu DMC z pomiarem zakłócenia działającego na otrzymanym modelu stanowiska grzejąco-chłodzącego

Stanowisko grzejąco-chłodzące jest obiektem bardzo wolnym, dlatego aby móc szybko znaleźć zadowalające parametry regulatora, najpierw należy wykonać jego strojenie na modelu za pomocą symulacji. W tym celu zaimplementowano funkcję `dmc_with_disturbance_heating_station()`:

Listing 2.9: Funkcja `dmc_with_disturbance_heating_station()`

```
function [y, u] = dmc_with_disturbance_heating_station(start, kend, ...
    N, Nu, D, lambda, Dz, y_set_time, y_set_value, z, consider_disturbance)

%% Configuration
addpath("approximation\");

% Controls limits
du_min = -100;
du_max = 100;
u_min = 0;
u_max = 100;

%% Step responses

working_point_u = 26;
```

```

working_point_z = 0;
step_value_u = 45;
step_value_z = 20;

% Input - output path
name1 = "data/l2_step_value=" + string(step_value_u) + ".csv";
raw_data1 = load(name1);
Ypp = raw_data1(1);
Upp = working_point_u;

[xopt1, td1] = approximation(step_value_u, working_point_u, raw_data1);
K1 = xopt1(1);
T11 = xopt1(2);
T21 = xopt1(3);

s = step_response(0, 0, D+1, K1, T11, T21, td1);
s = s(2:end);
[a1, b1] = calculate_coefficients(T11, T21, K1);

% Disturbance - output path
name2 = "data/zad2_disturbance2_step=" + string(step_value_z) + ".csv";
raw_data2 = load(name2);
Yzpp = raw_data2(1);

[xopt2, td2] = approximation(step_value_z, working_point_z, raw_data2);
K2 = xopt2(1);
T12 = xopt2(2);
T22 = xopt2(3);

sz = step_response(0, 0, Dz+1, K2, T12, T22, td2);
sz = sz(2:end);
[a2, b2] = calculate_coefficients(T12, T22, K2);

%% Algorithm

% Fill M matrix
M = zeros(N, Nu);
for i=1:Nu
    M(i:end,i)=s(1:N-i + 1);
end

% Fill MP matrix
MP = zeros(N, D-1);
for i = 1:N
    for j = 1:D-1
        if i+j <= D
            MP(i, j) = s(i+j) - s(j);
        else
            MP(i, j) = s(D) - s(j);
        end
    end
end

% Fill MZP matrix
MZP = zeros(N, Dz);
for j = 1:Dz
    for i=1:N
        if j == 1
            if i <= Dz
                MZP(i, j) = sz(i);
            else
                MZP(i, j) = sz(Dz);
            end
        end
    end
end

```

```

        else
            if i + j - 1 <= Dz
                MZP(i, j) = sz(i+j-1) - sz(j - 1);
            else
                MZP(i, j) = sz(Dz) - sz(j - 1);
            end
        end
    end
end

% Regulator parameters
I = eye(Nu);
K = ((M'*M+lambda*I)^(-1))*M';
Ku = K(1,:)*MP;
Ke = sum(K(1, :));

% Variables initialization
y_u = ones(kend, 1) * Ypp;
y_z = ones(kend, 1) * Yzpp;
y = ones(kend, 1) * Ypp;
u = ones(kend, 1) * Upp;
deltauk_p = zeros(D-1, 1);
deltaz_p = zeros(Dz, 1);

y_zad(1:y_set_time) = Ypp;
y_zad(y_set_time:kend) = y_set_value;

accumulated_error = 0;

% Main loop
for k=start:kend
    % Generate process output
    y_u(k) = Ypp + heating_station_simulation(u(k-td1-1) - Upp, ...
        u(k-td1-2) - Upp, y_u(k-1) - Ypp, y_u(k-2) - Ypp, a1, b1);
    y_z(k) = Yzpp + heating_station_simulation(z(k-td2-1), ...
        z(k-td2-2), y_z(k-1) - Yzpp, y_z(k-2) - Yzpp, a2, b2);
    y(k) = y_u(k) + y_z(k) - Yzpp;

    % Compute error
    ek = y_zad(k) - y(k);
    accumulated_error = accumulated_error + ek^2;

    % Disturbance effect
    z_part = 0;
    for j=0:(Dz-1)
        z_part = z_part + K(1, :)*MZP(:, j+1)*deltaz_p(1+j);
    end

    % Compute deltau variable for given control horizon
    if consider_disturbance
        deltauk = Ke*ek-Ku*deltauk_p - z_part;
    else
        deltauk = Ke*ek-Ku*deltauk_p;
    end

    % Back deltau window
    for n=D-1:-1:2
        deltauk_p(n) = deltauk_p(n-1);
    end

    % Calculate deltaz window
    for n=Dz:-1:2
        deltaz_p(n) = deltaz_p(n-1);
    end
end

```

```

end

if(deltauk < du_min)
    deltauk = du_min;
elseif(deltauk > du_max)
    deltauk = du_max;
end

% Manipulate variable for time k
u(k) = u(k-1) + deltauk;

if(u(k) < u_min)
    u(k) = u_min;
elseif(u(k) > u_max)
    u(k) = u_max;
end

% Delta u for time k
deltauk_p(1) = u(k) - u(k-1);
deltaz_p(1) = z(k) - z(k-1);
end

fprintf("Error: %f\r\n", accumulated_error);
end

```

Funkcja ta jako argumenty przyjmuje moment początkowy (**start**) i końcowy (**kend**) symulacji, parametry N , N_u , D , λ , D_z regulatora DMC, moment skoku wartości zadanej (**y_set_time**) oraz jego wartość (**y_set_value**), wektor zakłóceń (**z**) oraz parametr mówiący, czy regulator ma uwzględniać zmierzone zakłócenia, czy nie (**consider_disturbance**).

Funkcja ta nie różni się za wiele od funkcji o podobnej nazwie (**dmc_with_disturbance()**) napisanej w ramach projektu przedlaboratoryjnego. Jediną różnicą jest to, że regulator operuje tutaj na modelu stanowiska grzejąco-chłodzącego. Na początku funkcji, po wprowadzeniu wszystkich parametrów, są odczytywane z plików CSV dane zebrane podczas laboratorium. Na tej podstawie dokonywana jest aproksymacja oraz wyznaczenie wektorów współczynników odpowiedzi skokowej **s** oraz **sz**. Następnie wykonywany jest właściwy algorytm DMC, który został już opisany w części projektowej niniejszego sprawozdania. Różnica pojawia się w pętli **for**, gdzie jako wartość wyjściową obiektu jest brane wyjście modelu. Istotne jest to, że obiekt ten posiada tor wejście $\rightarrow$ wyjście oraz zakłócenia $\rightarrow$ wyjście, a więc funkcja **heating_station_simulation()** jest wykonywana dwa razy: raz dla sygnału sterującego, raz dla zakłóceń, a następnie ich wyniki są sumowane przy odpowiednim uwzględnieniu warunków początkowych.

Aby przetestować powyższy algorytm regulacji napisano skrypt 2.10 - **dmc_with_disturbance_test.m**:

Listing 2.10: Skrypt do symulowania działania regulatora DMC z pomiarem zakłócenia na podstawie modelu

```

clear;

% Paramaters
start = 20;
kend = 600;
N = 50;
Nu = 6;
D = 400;
Dz = 400;
lambda = 0.01;

% Set value parameters

```

```

set_value = 45;
set_time = 50;

% Disturbance parameters
z_start = 300;
z_step = 30;
z = zeros(kend, 1);
z(z_start:kend) = z_step;

consider_disturbance = true;

% Simulation
[y, u] = dmc_with_disturbance_heating_station(start, kend, N, Nu, D, ...
    lambda, Dz, set_time, set_value, z, consider_disturbance);

% Plot graphs
y_zad(1:set_time) = y(1);
y_zad(set_time:kend) = set_value;
len = length(y);

figure;
stairs(1:len, y);
hold on;
stairs(1:len, y_zad, '--');
xline(z_start, 'r--');
title("Wyjście procesu");
xlabel("k");
ylabel("Y / Y_{zad}");
hold off;

figure;
stairs(u);
xline(z_start, 'r--');
title("Sygnał sterujący");
xlabel("k");
ylabel("u");

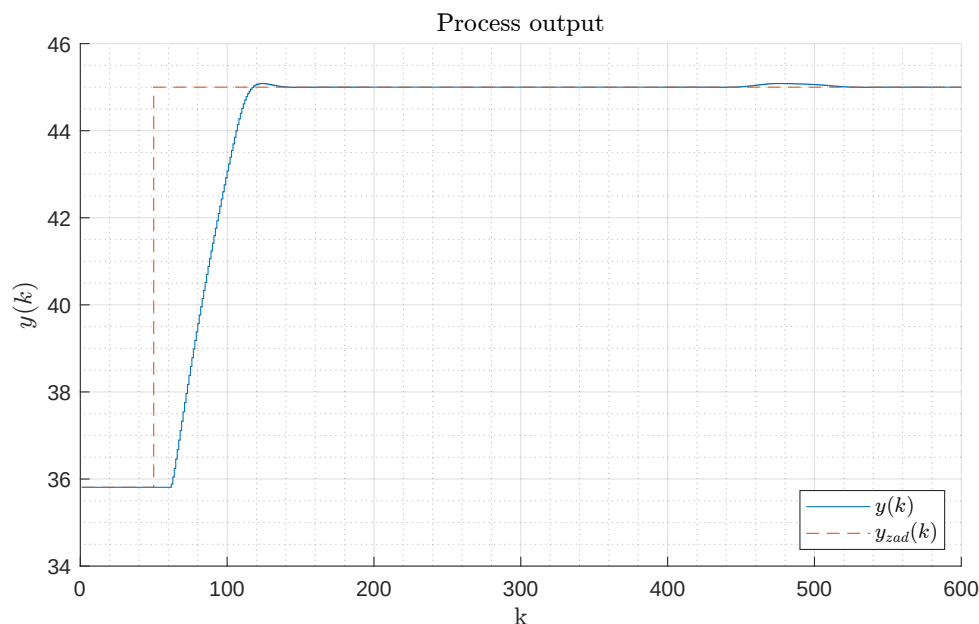
```

2.2.2. Testowanie i strojenie regulatora DMC z pomiarem zakłócenia na otrzymanym modelu stanowiska grzejąco-chłodzącego

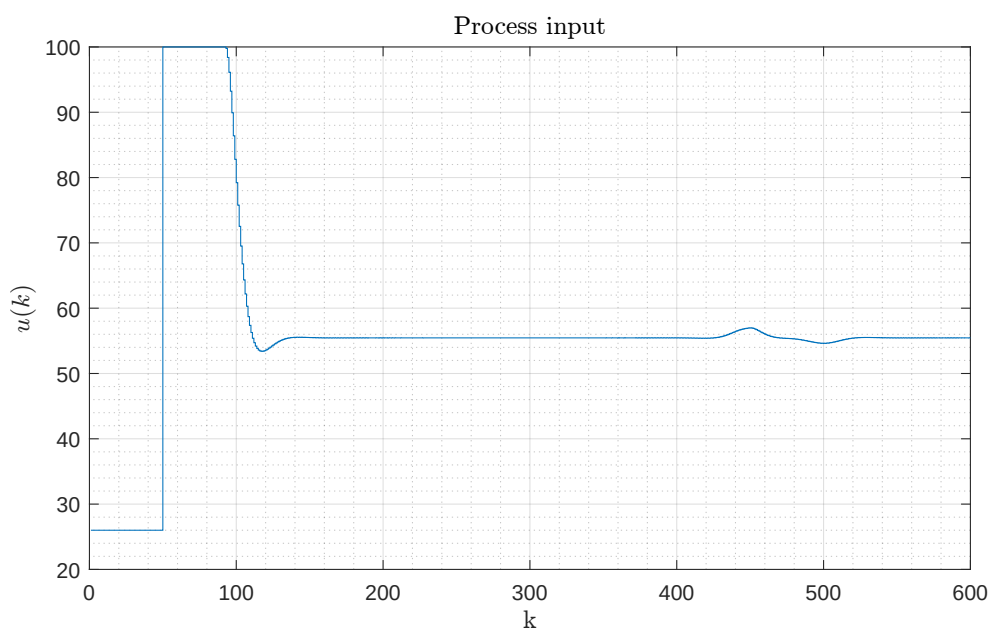
Strojenie i testowanie przy braku zakłócenia

Na początku dokonano strojenia regulatora na modelu działającym bez jakichkolwiek zakłóceń. Proces strojenia rozpoczęto od symulacji pracy DMC dla parametrów $D = N = N_u = 400$, $\lambda = 0,01$. Przyjęto taką wartość horyzontu dynamiki, gdyż w poprzednim bloku laboratoryjnym stwierdzono, że stan obiektu dochodzi bardzo blisko stanu ustalonego w tym czasie dla każdego wykonywanego skoku sterowania u . Na wykresie 2.8 widać, że jest to kompromis między ilością wykonywanych obliczeń (im większe horyzonty, tym więcej obliczeń jest potrzebnych w każdej iteracji), a idealnym osiągnięciem stanu ustalonego. Dalsze zmniejszanie parametru D nie ma sensu, gdyż wtedy wchodzi się w obszar, gdzie odpowiedź obiektu nie zdąży się ustalić.

Wykresy 2.9 oraz 2.10 przedstawiają wyjście obiektu $y(k)$ oraz wartość sterowania $u(k)$ dla parametrów DMC podanych powyżej. Suma kwadratów błędów wyniosła w tym przypadku 2324,960686. W okolicach 450 iteracji k widać delikatne zafalowanie na przebiegach. Wynika to z przyjętego parametru D . Jednak zwiększenie horyzontu dynamiki nie przynosiło w tym przypadku satysfakcjonujących rezultatów ze względu na ograniczoną ilość czasu, przez który wykonywano identyfikację obiektu za pomocą skoku wartości sterowania. Natomiast zmniejszenie wartości D powodowało znaczące pogorszenie się pracy regulatora w otoczeniu $k = 450$.

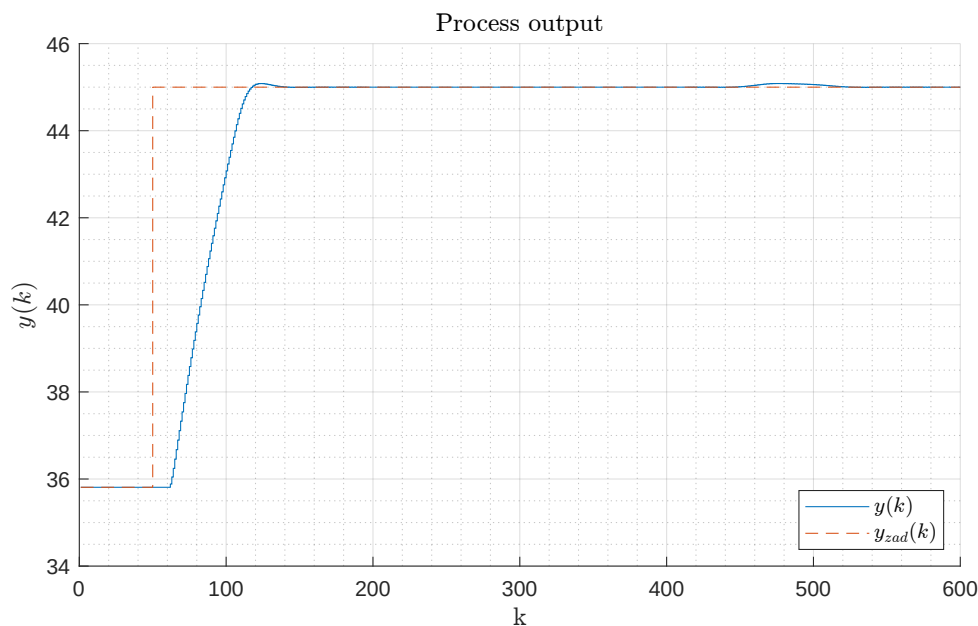


Rys. 2.9: Wyjście obiektu $y(k)$ przy zerowych zakłóceniach dla regulatora DMC ($D = 400$, $N = 400$, $N_u = 400$, $\lambda = 0,01$)

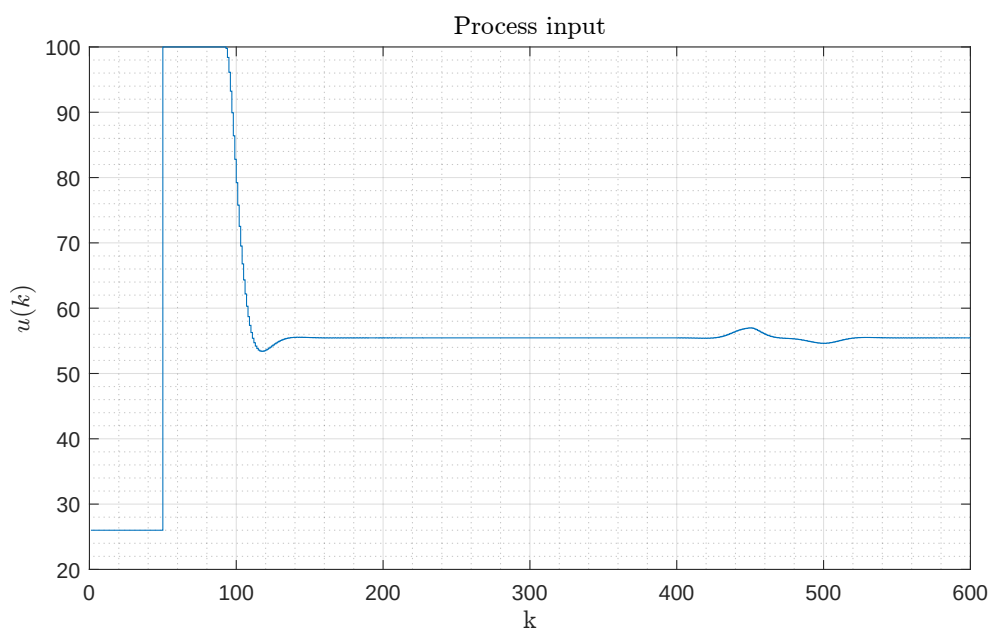


Rys. 2.10: Sterowanie obiektu $u(k)$ przy zerowych zakłóceniach dla regulatora DMC ($D = 400$, $N = 400$, $N_u = 400$, $\lambda = 0,01$)

Na wykresach 2.11 i 2.12 przedstawiono przebiegi $y(k)$ oraz $u(k)$ dla $D = 400$ oraz zmniejszonych $N = N_u = 50$ przy zachowanym $\lambda = 0,01$. Była to granica, poniżej której zauważalne było już pogorszenie jakości regulacji. Dla tych parametrów błąd wyniósł 2324,960478, czyli w zasadzie tyle samo co ostatnio. Jednak warto zauważyć, że ta miara jakości nie będzie się mocno różniła dla regulatora, którego empirycznie można by było ocenić jako dobry, od tego, którego można by było ocenić jako nie najlepszy. Wynika to z charakteru obiektu, który ma duże opóźnienie i największa część błędu zależy właśnie od niego. Drobne oscylacje lub przeregulowania mają mniejszy wpływ na ostateczny wynik tej miary jakości.



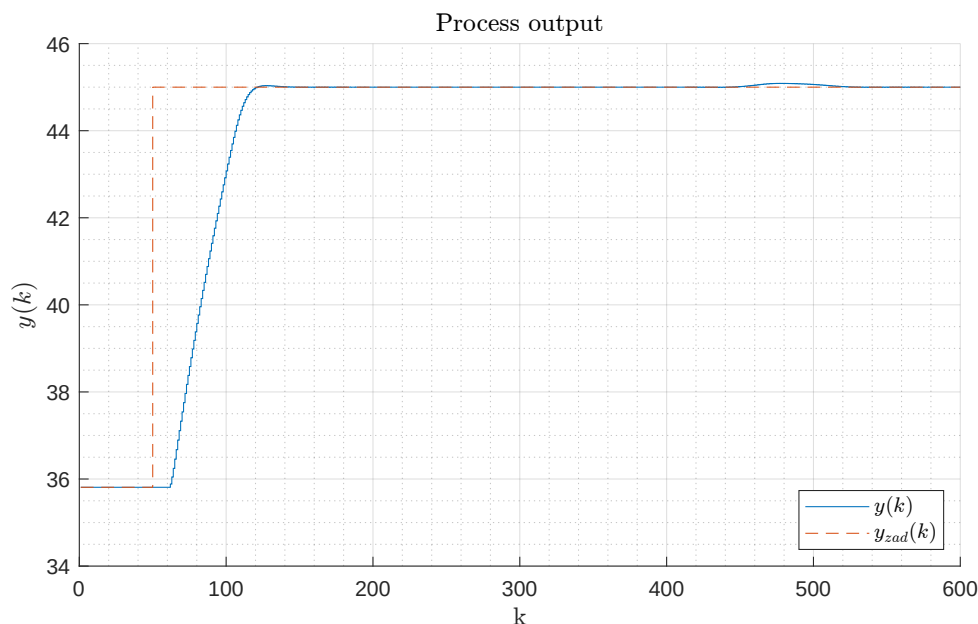
Rys. 2.11: Wyjście obiektu $y(k)$ przy zerowych zakłóceniach dla regulatora DMC ($D = 400$, $N = 50$, $N_u = 50$, $\lambda = 0,01$)



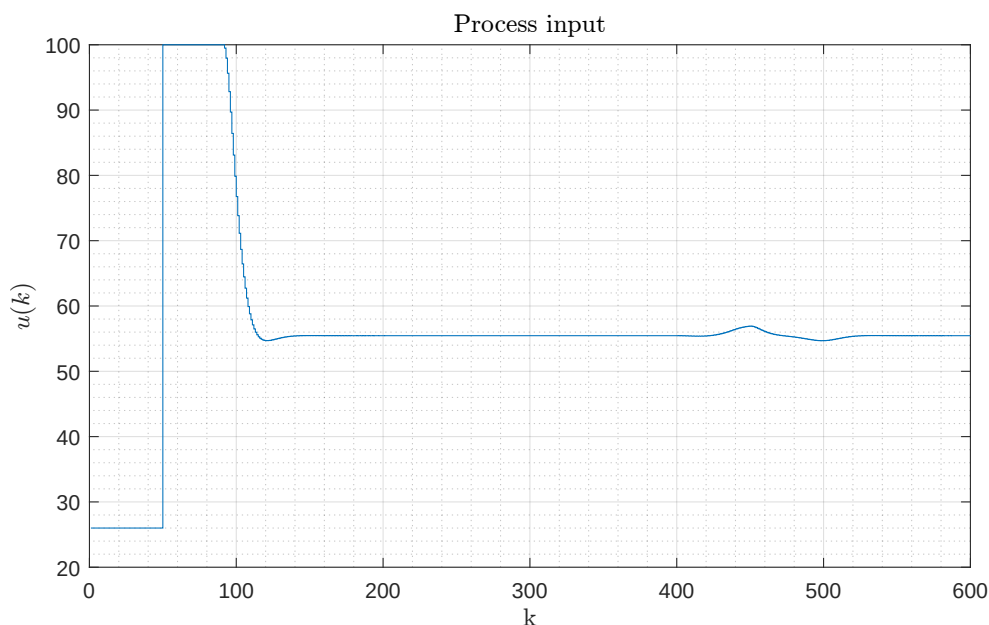
Rys. 2.12: Sterowanie obiektu $u(k)$ przy zerowych zakłóceniach dla regulatora DMC ($D = 400$, $N = 50$, $N_u = 50$, $\lambda = 0,01$)

Na wykresach 2.13 i 2.14 pokazano pracę dostrojonego regulatora. Zmniejszono N_u do 6, gdyż była to granica, poniżej której działanie regulatora się pogarszało. Zachowano taką samą wartość $\lambda = 0,01$, gdyż jest to bardzo dobry kompromis między szybkością regulacji a jej jakością objawiającą się na braku przeregulowań.

W ten sposób osiągnięto bardzo dobrze działający regulator, jednocześnie zmniejszywszy horyzonty predykcji i sterowania na tyle, ile to możliwe, aby zachować dobrą jakość regulacji i jednocześnie nie wykonywać zbędnych obliczeń.



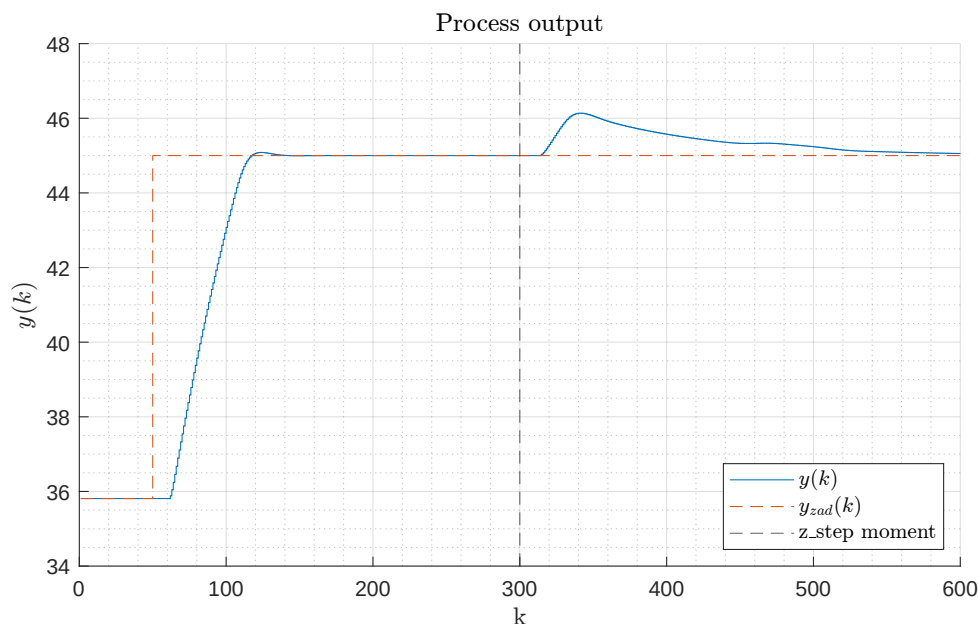
Rys. 2.13: Wyjście obiektu $y(k)$ przy zerowych zakłóceniach dla regulatora DMC ($D = 400$, $N = 50$, $N_u = 6$, $\lambda = 0,01$)



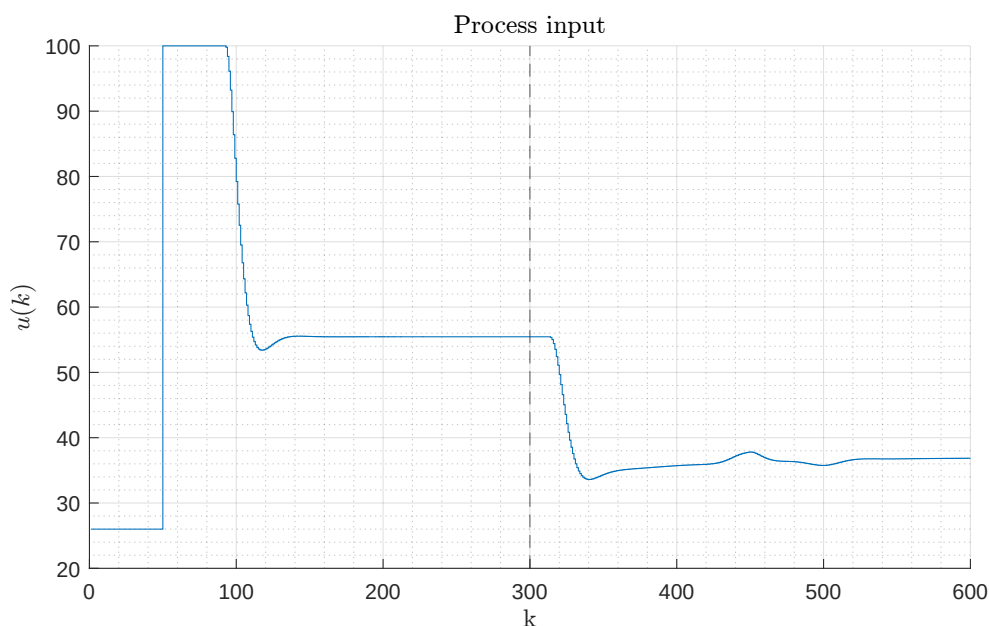
Rys. 2.14: Sterowanie obiektu $u(k)$ przy zerowych zakłóceniach dla regulatora DMC ($D = 400$, $N = 50$, $N_u = 6$, $\lambda = 0,01$)

Testowanie przy skokowo zmieniającym się zakłóceniu

W doświadczeniu tym dodano do sygnału wyjściowego skok zakłócenia mierzalnego w chwili $k = 300$ o wartości $\Delta z = 30$. Wybrano horyzont dynamiki dla toru zakłócenie $\rightarrow$ wyjście $D_z = 400$, czyli taki sam jak wartość D , gdyż z wykresu 2.5 wynika, że w przybliżeniu tyle ustala się odpowiedź obiektu po wykonaniu skoku zakłócenia. W funkcji do aproksymacji odpowiedzi skokowej dla zakłócenia wybrano dane z pliku CSV wykonanego dla skoku `z.step_value=20`, gdyż dla tej wartości inne zakłócenia oraz szum pomiarowy nie mają już takiego wpływu, co przekłada się na dobrą jakość aproksymacji.



Rys. 2.15: Wyjście obiektu $y(k)$ przy dodanym zakłóceniu $\Delta z = 30$ w chwili $k = 300$ dla regulatora DMC nieuwzględniającego zakłóceń ($D = 400$, $N = 50$, $N_u = 6$, $\lambda = 0,01$)

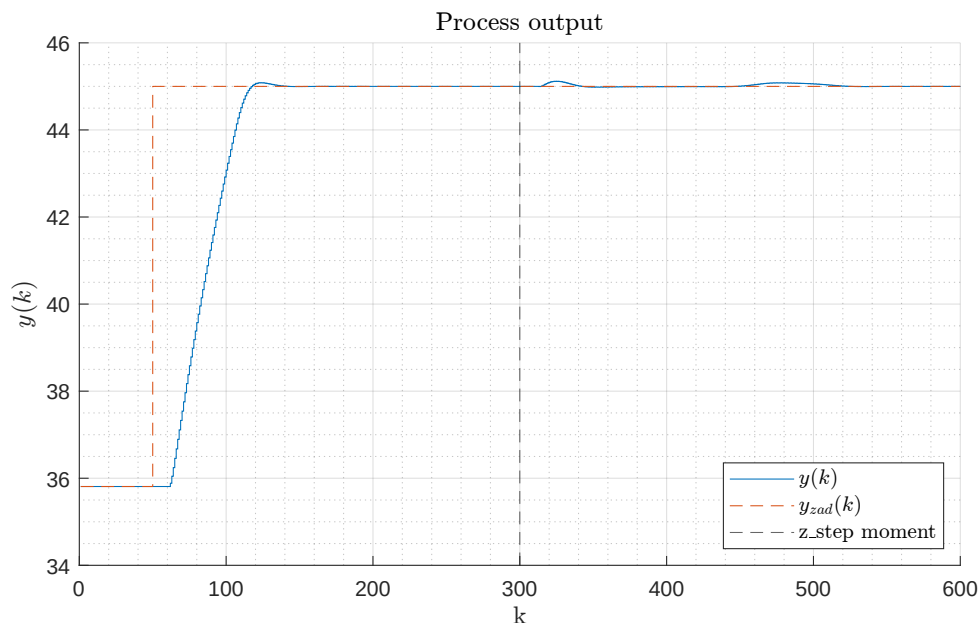


Rys. 2.16: Sterowanie obiektu $u(k)$ przy dodanym zakłóceniu $\Delta z = 30$ w chwili $k = 300$ dla regulatora DMC nieuwzględniającego zakłóceń ($D = 400$, $N = 50$, $N_u = 6$, $\lambda = 0,01$)

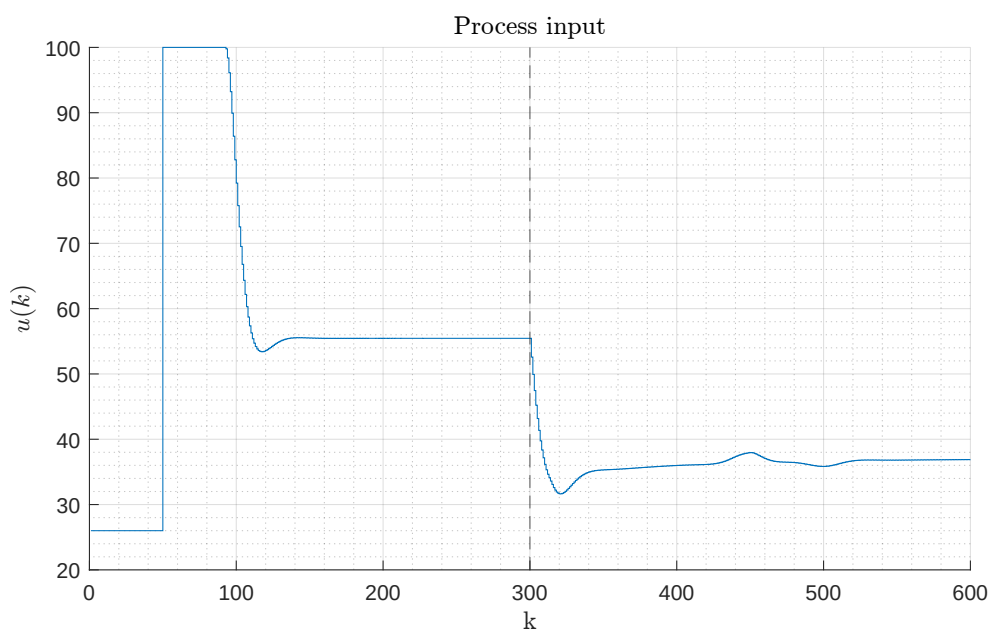
Jak widać na wykresach 2.15 oraz 2.16, jakość regulacji uległa pogorszeniu. Regulator nie mający informacji o zakłóceniu zaczyna reagować zmianą $u(k)$ na nieoczekiwaną zmianę wyjścia $y(k)$ spowodowaną zakłóceniem. Jednak ze względu na opóźnienie obiektu reakcja ta jest zbyt wolna, co powoduje skoki wartości $y(k)$ w momencie skokowej zmiany zakłóceń. W tym przypadku błąd regulacji wzrósł do 2998,919138.

Wykresy 2.17 i 2.18 przedstawiają działanie regulatora DMC przy takiej samej zmianie zakłócenia, jednak z włączoną opcją uwzględniania tego czynnika. Regulator w tym przypadku pracuje bardzo dobrze, na wyjściu obiektu prawie nie widać momentu, w którym pojawiło się

zakłócenie. Na wykresie 2.18 przedstawiającym zależność $u(k)$ widać, że wartość sterowania zmieniła się od razu po wykryciu zmiany zakłócenia, a nie z opóźnieniem, jak to miało miejsce na wykresie 2.16. Tym razem błąd regulacji wyniósł 2325,120308, czyli prawie tyle, co w przypadku braku zakłócenia.



Rys. 2.17: Wyjście obiektu $y(k)$ przy dodanym zakłóceniu $\Delta z = 30$ w chwili $k = 300$ dla regulatora DMC uwzględniającego zakłócenie ($D = 400$, $N = 50$, $N_u = 6$, $\lambda = 0,01$, $D_z = 400$)



Rys. 2.18: Sterowanie obiektu $u(k)$ przy dodanym zakłóceniu $\Delta z = 30$ w chwili $k = 300$ dla regulatora DMC uwzględniającego zakłócenie ($D = 400$, $N = 50$, $N_u = 6$, $\lambda = 0,01$, $D_z = 400$)

2.2.3. Implementacja algorytmu DMC z pomiarem zakłócenia działającego na rzeczywistym stanowisku laboratoryjnym

Implementacja algorytmu DMC z pomiarem zakłócenia nie różni się od tej opisanej w punkcie 2.2.1. Jedyńa różnica polega na nieużywaniu funkcji `dmc_with_disturbance_heating_station()`, gdyż większość jej zawartości została przeniesiona do głównej części programu, umożliwiając wygodniejsze wysyłanie sterowań do stanowiska grzejąco-chłodzącego oraz dodanie akwizycji danych. Napisany skrypt 2.11 - `dmc_disturbance_lab.m` jest implementacją regulatora na potrzeby tego laboratorium:

Listing 2.11: Regulator DMC z pomiarem zakłócenia używany do sterowania rzeczywistym obiektem

```
clear;

%% Configuration
addpath("approximation\");

addpath('D:\SerialCommunication');
initSerialControl COM4
sendControls(1, 50);

% Controls limits
du_min = -100;
du_max = 100;
u_min = 0;
u_max = 100;

% Parameters
start = 20;
kend = 600;
N = 50;
Nu = 6;
D = 400;
Dz = 400;
lambda = 0.01;
y_set_value = 45;
u_set_time = 20;
consider_disturbance = true;

% Disturbance trajectory
z_time_step_1 = 250;
z_time_step_2 = 425;
z_step_1 = 30;
z_step_2 = 10;

z = zeros(kend, 1);
z(z_time_step_1:z_time_step_2) = z_step_1;
z(z_time_step_2:kend) = z_step_2;

%% Step responses
working_point_u = 26;
working_point_z = 0;
step_value_u = 45;
step_value_z_approx = 20;

% Input - output path
name1 = "data/l2_step_value=" + string(step_value_u) + ".csv";
raw_data1 = load(name1);
Ypp = 34.1;
Upp = working_point_u;
```

```

[xopt1, td1] = approximation(step_value_u, working_point_u, raw_data1);
K1 = xopt1(1);
T11 = xopt1(2);
T21 = xopt1(3);

s = step_response(0, 0, D+1, K1, T11, T21, td1);
s = s(2:end);
[a1, b1] = calculate_coefficients(T11, T21, K1);

% Disturbance - output path
name2 = "data/zad2_disturbance2_step=" + string(step_value_z_approx) + ".csv";
raw_data2 = load(name2);
Yzpp = raw_data2(1);

[xopt2, td2] = approximation(step_value_z_approx, working_point_z, raw_data2);
K2 = xopt2(1);
T12 = xopt2(2);
T22 = xopt2(3);

sz = step_response(0, 0, Dz+1, K2, T12, T22, td2);
sz = sz(2:end);
[a2, b2] = calculate_coefficients(T12, T22, K2);

% Variables initialization
y_u = ones(kend, 1) * Ypp;
y_z = ones(kend, 1) * Yzpp;
y = ones(kend, 1) * Ypp;
u = ones(kend, 1) * Upp;
deltauk_p = zeros(D-1, 1);
deltaz_p = zeros(Dz, 1);

y_zad(1:u_set_time) = Ypp;
y_zad(u_set_time:kend) = y_set_value;

%% Plots and saving to file
% File name to acquired data
test_number = 12;
name = "data/DMC_object_" + string(test_number) + ".csv";

% Preparing files
file_id = fopen(name, 'a');
fprintf(file_id, "u(k), y(k)\n");

buffer_size = 15;
buffer = zeros(buffer_size, 2);
buffer_index = 1;

% Make window with plots
figure;
hold on;
h_y = stairs(1:start, y(1:start), 'DisplayName', 'y');
h_y_zad = stairs(1:start, y_zad(1:start), 'DisplayName', 'y\_zad');
hold off;

title("Wykres sterowania i odpowiedzi stanowiska grzewczego");
xlabel("Iteracje [k]");
ylabel("Wartości");
legend;
grid on;

figure;
hold on;
h_u = stairs(1:start, u(1:start), 'DisplayName', 'u');

```

```

hold off;

title("Wykres sterowania i odpowiedzi stanowiska grzewczego");
xlabel("Iteracje [k]");
ylabel("Wartości");
legend;
grid on;

%% Algorithm

% Fill M matrix
M = zeros(N, Nu);
for i=1:Nu
    M(i:end,i)=s(1:N-i + 1);
end

% Fill MP matrix
MP = zeros(N, D-1);
for i = 1:N
    for j = 1:D-1
        if i+j <= D
            MP(i, j) = s(i+j) - s(j);
        else
            MP(i, j) = s(D) - s(j);
        end
    end
end

% Fill MZP matrix
MZP = zeros(N, Dz);
for j = 1:Dz
    for i=1:N
        if j == 1
            if i <= Dz
                MZP(i, j) = sz(i);
            else
                MZP(i, j) = sz(Dz);
            end
        else
            if i + j - 1 <= Dz
                MZP(i, j) = sz(i+j-1) - sz(j - 1);
            else
                MZP(i, j) = sz(Dz) - sz(j - 1);
            end
        end
    end
end

% Regulator parameters
I = eye(Nu);
K = ((M'*M+lambda*I)^(-1))*M';
Ku = K(1,:)*MP;
Ke = sum(K(1, :));

accumulated_error = 0;

% Main loop
for k=start:kend
    % Process output
    y(k) = readMeasurements(1);

    % Compute error
    ek = y_zad(k) - y(k);

```

```

accumulated_error = accumulated_error + ek^2;

% Disturbance effect
z_part = 0;
for j=0:(Dz-1)
    z_part = z_part + K(1, :)*MZIP(:, j+1)*deltaz_p(1+j);
end

% Compute deltau variable for given control horizon
if consider_disturbance
    deltau_k = Ke*ek-Ku*deltau_k_p - z_part;
else
    deltau_k = Ke*ek-Ku*deltau_k_p;
end

% Back deltau window
for n=D-1:-1:2
    deltau_k_p(n) = deltau_k_p(n-1);
end

% Calculate deltaz window
for n=Dz:-1:2
    deltaz_p(n) = deltaz_p(n-1);
end

if(deltau_k < du_min)
    deltau_k = du_min;
elseif(deltau_k > du_max)
    deltau_k = du_max;
end

% Manipulate variable for time k
u(k) = u(k-1) + deltau_k;

if(u(k) < u_min)
    u(k) = u_min;
elseif(u(k) > u_max)
    u(k) = u_max;
end

% Delta u for time k
deltau_k_p(1) = u(k) - u(k-1);
deltaz_p(1) = z(k) - z(k-1);

% Send control command to the process
sendControlsToG1AndDisturbance(u(k), z(k));

% Add data to buffer
buffer(buffer_index, :) = [u(k), y(k)];
buffer_index = buffer_index + 1;

% Write data to file
if buffer_index > buffer_size
    fprintf(file_id, '%f, %f\n', buffer');
    buffer_index = 1;
end

% Update plots
set(h_u, 'XData', 1:k, 'YData', u(1:k));
set(h_y, 'XData', 1:k, 'YData', y(1:k));
set(h_y_zad, 'XData', 1:k, 'YData', y_zad(1:k));
drawnow;

```

```

        waitForNewIteration();
    end

    fclose(file_id);
    sendControlsToG1AndDisturbance(Upp, 0);

    fprintf("Error: %f\r\n", accumulated_error);

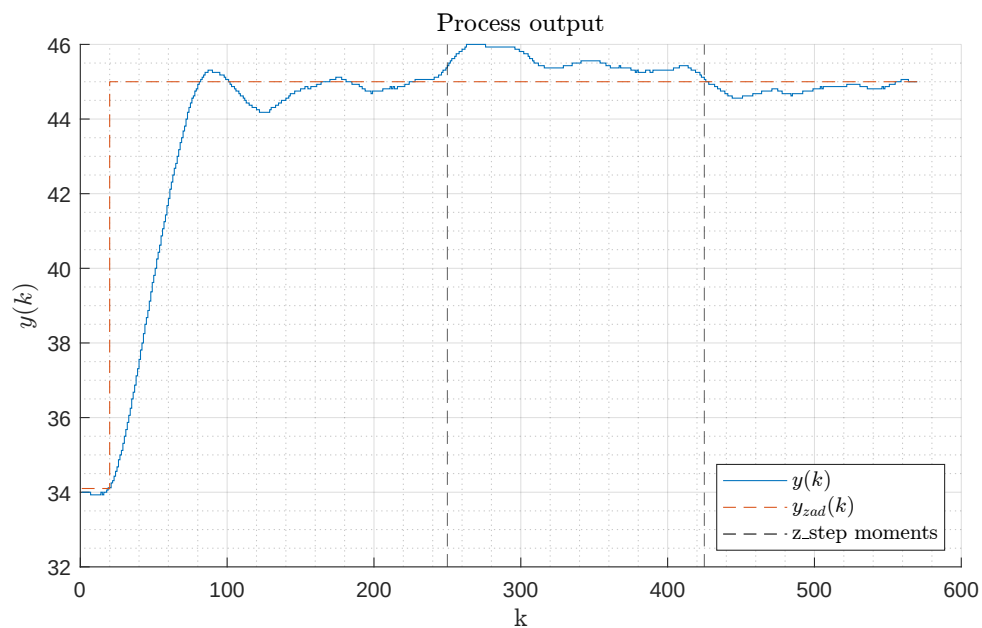
```

Na początku programu inicjalizowana jest komunikacja ze stanowiskiem grzejąco-chłodzącym. Następnie po podaniu parametrów regulatora i programu, wprowadzeniu wartości i momentów skoków y_{zad} oraz zakłócenia z , odbywa się odczyt plików CSV z danymi odpowiedzi skokowych stanowiska oraz otrzymywanie wektorów $\mathbf{s}$ oraz $\mathbf{sz}$ na tej samej zasadzie co wcześniej. W następnym kroku jest przygotowywany nowy plik CSV do akwizycji danych oraz wykresy, na których na bieżąco będą się pojawiały wartości sterowania i wyjścia procesu. Następnie rozpoczyna się implementacja właściwego algorytmu, która jest identyczna jak w opisanej wcześniej funkcji `dmc_with_disturbance_heating_station()`, z tą różnicą, że regulator nie bazuje już na modelu, lecz na danych odczynach ze stanowiska grzejąco-chłodzącego, a także dodatkowo jest wykonywana akwizycja danych oraz aktualizowanie informacji na wykresach.

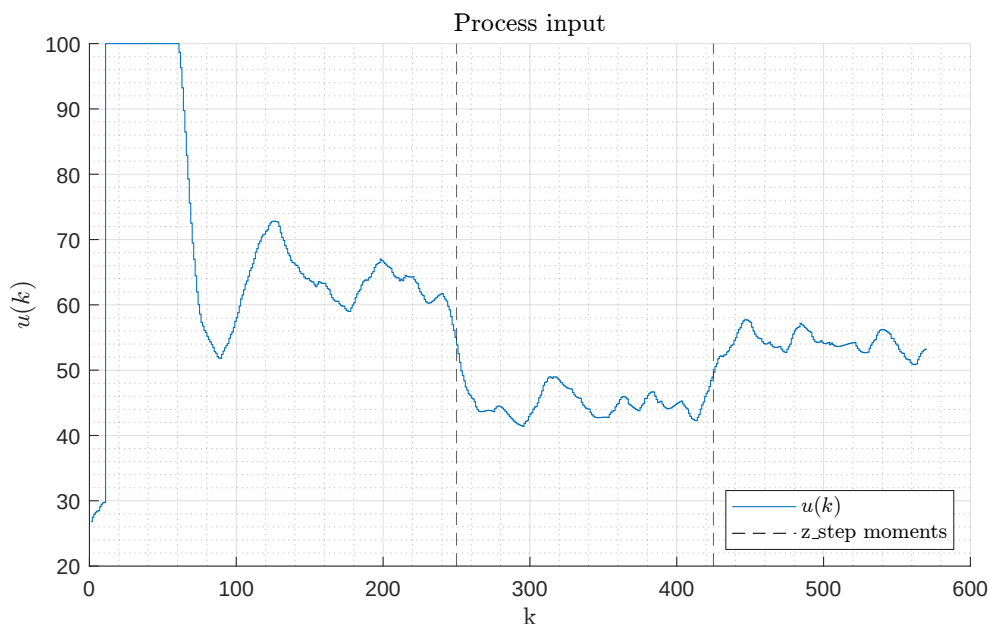
2.2.4. Testowanie regulatora DMC z pomiarem zakłócenia na rzeczywistym stanowisku laboratoryjnym

Testowanie regulatora nieuwzględniającego zakłócenia

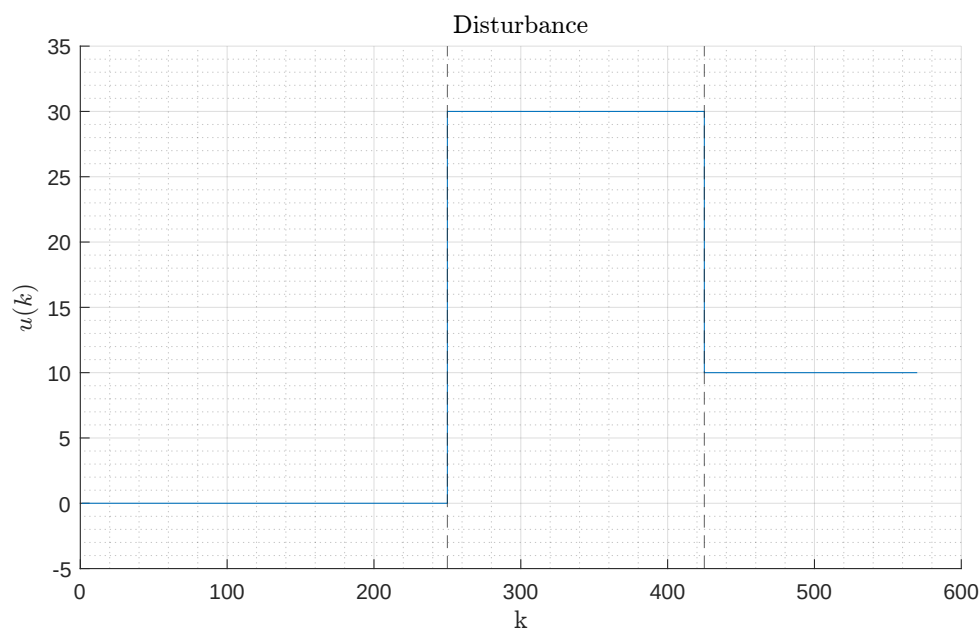
Testy rozpoczęto od wykonania skryptu 2.11 - `dmc_disturbance_lab.m` z ustawionym parametrem `consider_disturbance = false`. Parametry regulatora były takie same jak te wyznaczone z użyciem symulacji ($D = 400$, $N = 50$, $N_u = 6$, $\lambda = 0,01$, $D_z = 400$). Wykonano skok wartości zadanej, po czym dwa skoki zakłócenia, co widoczne jest na poniższych wykresach:



Rys. 2.19: Wyjście obiektu $y(k)$ przy skokowych zmianach zakłócenia dla regulatora DMC nieuwzględniającego zakłócenia ($D = 400$, $N = 50$, $N_u = 6$, $\lambda = 0,01$, $D_z = 400$)



Rys. 2.20: Sterowanie obiektu $u(k)$ przy skokowych zmianach zakłócenia dla regulatora DMC niewzględniającego zakłócenia ($D = 400$, $N = 50$, $N_u = 6$, $\lambda = 0,01$, $D_z = 400$)



Rys. 2.21: Zakłócenie dodane do procesu

Na pierwszy rzut oka widać, że jakość regulacji jest o wiele gorsza niż w przypadku symulacji wykonywanej w podpunkcie 2.2.2. Wynika to z tego, że stanowisko grzejąco-chłodzące jest rzeczywistym procesem, na który wpływają różne zakłócenia niemierzalne oraz liczny szum pomiarowy. Dodatkowo algorytm DMC jest bardzo wrażliwy na błędy modelu lub na zmiany parametrów modelu podczas pracy. W tym przypadku miało jedno i drugie - nie da się w obecności tak wielu zakłóceń zebrać danych pozwalających dokonać idealnej aproksymacji transmitancją dwuinercyjną. Można by było próbować wykonywać ten sam skok wiele razy i w ten sposób otrzymać wektor średnich współczynników odpowiedzi skokowej, którą dopiero wtedy należałoby poddać aproksymacji. Jednak i w tym podejściu pojawił się problem ze zmiennością punktów pracy, a przez to taki zabieg uśredniania kilku serii pomiarowych musiałby być złożo-

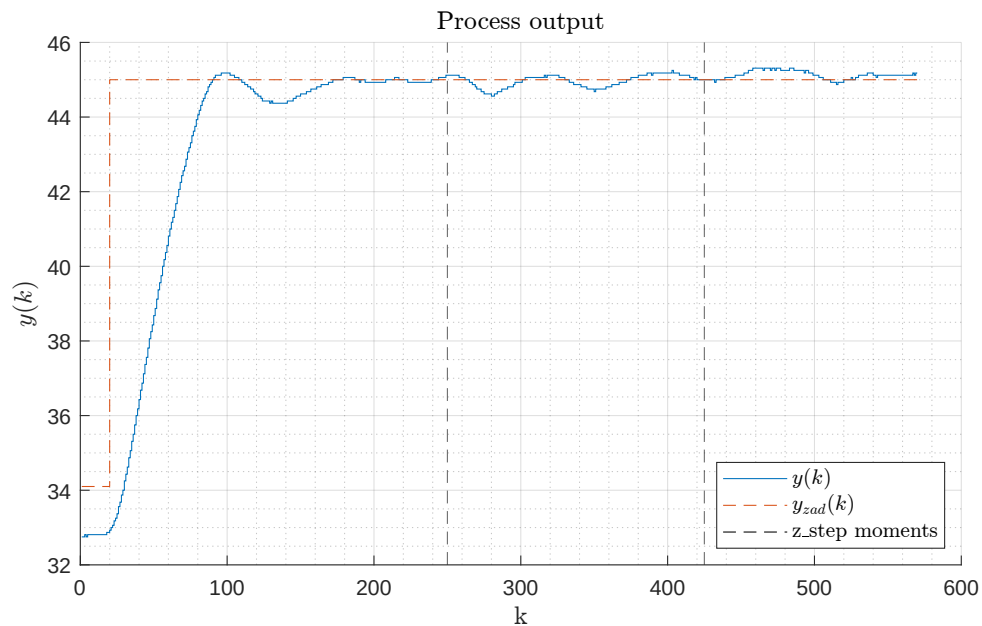
nym zagadnieniem. Tak więc w przypadku strojenia regulatora z użyciem symulacji regulator jak i symulacja bazują na tym samym modelu, co pozwala uzyskać bardzo dobrą jakość regulacji. Niestety w przypadku rzeczywistego stanowiska model, na którym bazuje algorytm, i obiekt będą się zawsze trochę różnić.

Dodatkowym problemem jest zmienność punktu pracy Y_{pp} spowodowana wzrostem temperatury w pracowni, co było wspomniane już wcześniej. Dlatego też za każdym razem wartość temperatury, do której udało się sprowadzić obiekt w stanie ustalonym, się różniła. Aby zmniejszyć ten efekt, przed każdym uruchomieniem regulatora DMC podawano jako parametr algorytmu zaktualizowany punkt początkowy Y_{pp} , co jednak nie zawsze dawało w pełni zadowalający efekt, gdyż w przypadku DMC niepoprawny punkt pracy może skutkować nie najlepszym działaniem regulatora także chwilę po dojściu $y(k)$ do otoczenia wartości zadanej.

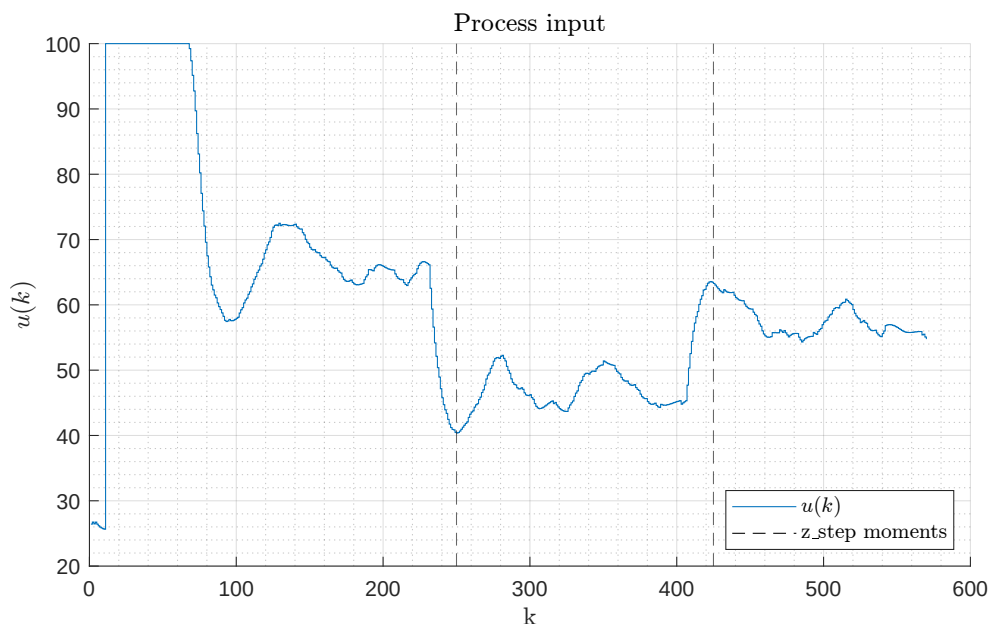
Na wykresie 2.19 w momencie skoku zakłócenia z wystąpił zauważalny skok wartości $y(k)$, jednak regulator poradził sobie z tym całkiem dobrze. Niestety liczne pozostałe zakłócenia sprawiają, że skok ten nie wyróżnia się aż tak bardzo na tle stanu ustalonego.

Testowanie regulatora uwzględniającego zakłócenie

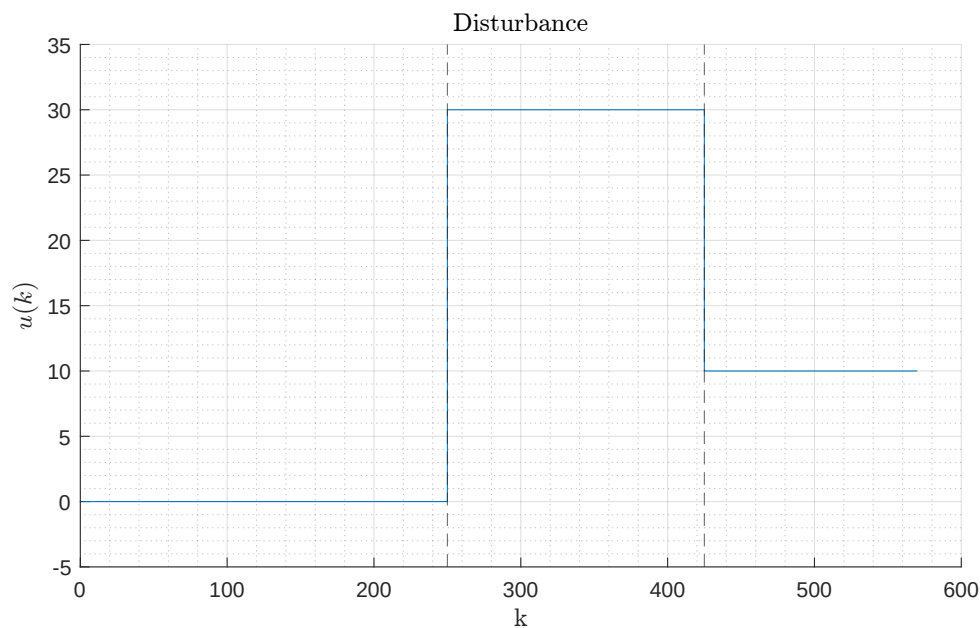
W tym przypadku zmieniono parametr `consider_disturbance` na `true`, przez co regulator uwzględniał w swoich obliczeniach zmierzoną wartość zakłócenia z . Po wykresie 2.22 od razu widać, że jakość regulacji się poprawiła. Na sygnale $y(k)$ w ogóle nie widać momentów, gdzie dokonywane były skoki zakłócenia. Jedynie cały czas wykres lekko oscyluje z losowym okresem wokół wartości zadanej. Wynika to głównie z zakłóceń. Te chwilowe uchyby nie przekraczają w zasadzie $0,5^\circ\text{C}$, a więc można powiedzieć, że w praktyce jakość regulacji jest bardzo dobra.



Rys. 2.22: Wyjście obiektu $y(k)$ przy skokowych zmianach zakłócenia dla regulatora DMC uwzględniającego zakłócenie ($D = 400$, $N = 50$, $N_u = 6$, $\lambda = 0,01$, $D_z = 400$)



Rys. 2.23: Sterowanie obiektu $u(k)$ przy skokowych zmianach zakłócenia dla regulatora DMC uwzględniającego zakłócenie ($D = 400$, $N = 50$, $N_u = 6$, $\lambda = 0,01$, $D_z = 400$)



Rys. 2.24: Zakłócenie dodane do procesu

2.3. Wnioski

Powyższe eksperymenty pokazały jak użyteczną właściwością jest zdolność algorytmu DMC do uwzględniania zakłóceń mierzalnych. Niestety nie każde zakłócenie jest takie, co w dużej mierze ogranicza możliwość zastosowania tego algorytmu w niektórych zastosowaniach. Regulatory DMC są dość podatne na zakłócenia niemierzalne, co może być dużym problemem. Wszystko zależy od konkretnego przypadku, dlatego warto najpierw, o ile to możliwe, zamodelować dany proces i przeprowadzić stosowne symulacje. Trzeba jednak pamiętać, że teoria i symulacje nie zawsze pokrywają się dobrze z rzeczywistością, co widoczne było także podczas tego ćwiczenia.